

Beluga: Block Synchronization for BFT Consensus Protocols

Abstract

Modern high-throughput BFT consensus protocols use streamlined push-pull mechanisms to disseminate blocks and keep happy-path performance optimal. Yet state-of-the-art designs lack a principled and efficient way to exchange blocks, which leaves them open to targeted attacks and performance collapse under network asynchrony. This work introduces the *block synchronizer*, a simple abstraction that drives incremental block retrieval and enforces resource-aware exchange. Its interface and role sit cleanly inside a modern BFT consensus stack. We also uncover a new attack, where an adversary steers honest validators into redundant, uncoordinated pulls that exhaust bandwidth and stall progress. Beluga is a modular and scarcity-aware instantiation of the block synchronizer. It achieves optimal common-case latency while bounding the cost of recovery under faults and adversarial behavior. We integrate Beluga into Mysticeti, the consensus core of the Sui blockchain, and show on a geo-distributed AWS deployment that Beluga sustains optimal performance in the optimistic path and, under attack, delivers up to 3× higher throughput and 25× lower latency than prior designs. The Sui blockchain adopted Beluga in production.

CCS Concepts

• Security and privacy → Distributed systems security.

Keywords

Byzantine Fault-tolerant, Block Synchronization, BFT Consensus

ACM Reference Format:

. 2018. Beluga: Block Synchronization for BFT Consensus Protocols. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 20 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

The last decade of research in high-throughput Byzantine fault-tolerant (BFT) consensus protocols has unveiled that achieving world-class performance requires two key design choices. Firstly, modern blockchains decouple block ordering from bulk data dissemination [16, 51], and secondly, they chain the disseminated data with causal references to past disseminated data [10, 14, 16]. Both properties depart from the blueprint used by legacy protocols such as PBFT [13], where data is disseminated solely by a rotating leader using monotonically increasing view numbers; instead, they adopt

a concurrent data dissemination approach. This means each validator is expected to assemble transactions into blocks or batches and broadcast them to all other validators [16].

This dissemination usually imposes chaining or causal dependencies between blocks, but protocols implement this with varying degrees of rigidity. Some protocols, such as Narwhal-Hotstuff [16] and Autobahn [24] achieve this through direct dissemination, after which a separate module imposes a causal order over this data. Others, such as HashGraph [25] and Blocklace-based systems [4, 34] have validators assemble transactions into a block and disseminate the block by referencing as many blocks as possible. Finally, protocols such as Bullshark [43] and Mysticeti [6] force validators to only disseminate blocks upon collecting at least a quorum of blocks from the previous round. Regardless of the approach, to implement atomic broadcast [11], they must implement the reliable broadcast abstraction. Specifically, even if the BFT consensus protocol ensures consistency, the dissemination layer must still enforce a liveness-relevant property—totality [11]: if one honest validator receives a block, then all other validators must be able to obtain it as well.

Through manual inspection of numerous high-performant BFT codebases [19, 23, 27, 29, 39–41, 47, 50], we observed that contrary to the liveness proofs and descriptions of these protocols, none of the state-of-the-art protocols actually implement an upfront reliable broadcast, such as Bracha broadcast [9]. That is, they do not implement the totality property through a double-echo. The reason is clear: this would be prohibitively expensive as it consumes precious bandwidth, and the double-echo is rarely needed (only upon faults or poor network conditions) [16, 35]. We empirically uncover the *same* implicit two-phase pattern: an optimistic *push* of block identifiers followed by a probabilistic *pull* for missing blocks. We confirm this through a systematic inspection of diverse production and prototype codebases of state-of-the-art protocols, examining dissemination modules, recovery loops, timers, and traces. The optimistic push is implemented through weaker primitives such as Byzantine consistent broadcast (lacking *totality*) [30, 38, 41] or even best-effort broadcast [27, 29]. A recovery mechanism asynchronously *pulls* any missing blocks deemed “useful”.

The missing component. Despite its importance, this push-pull mechanism remains unstructured glue code, often involving uncoordinated pre-dissemination of block identifiers, timer-driven random pulls, and no explicit bounds on recovery message complexity (Sec. 2.2). Protocols run ad hoc logic on received blocks to determine whether and when to trigger the pull part of the protocol. This hidden component governs throughput under load, tail latency, and resilience against adversarial behavior and network conditions. Yet it lacks a specification or provable bounds.

As an almost expected consequence of this discrepancy between theory and practice, we uncover how an adversary can induce a targeted performance degradation that we call the *pull induction attack*. Byzantine replicas selectively withhold messages during initial dissemination so that only a small subset of honest validators receive a block. This then triggers redundant and overlapping pull requests from the remaining honest validators. The attack can

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference acronym 'XX, Woodstock, NY

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2018/06
<https://doi.org/XXXXXXX.XXXXXXX>

be repeated every round and can drastically increase latency and consume recovery bandwidth (Sec. 3). We even observe that some implementations entirely omit the pull mechanism, thus claiming happy path performance while silently failing to achieve liveness under adversarial settings.

In this work, we formalize the module of BFT consensus responsible for block synchronization, which we call the *block synchronizer* (Sec. 2.1). This module decides how validators push their blocks to other validators, how to fetch any missing causal dependencies, and runs a principled admission control on whether to include the block in its local dependency graph (i.e., use it as a parent). An ideal block synchronizer module must satisfy two main goals: (G1) optimal push latency along the optimistic path; and (G2) bounded amplification even under adversarial scheduling. Existing block synchronizer protocols fall short on G1 due to conservative multi-step push procedures (such as explicit consistent broadcast) and on G2 because of ad hoc push-pull designs. We provide a summary of these protocols in Sec. 2.2 (Table 1).

We then present Beluga, a block synchronizer module that satisfies all the goals above (Sec. 4). Beluga is modular and integrates into existing protocols without altering their safety guarantees or ordering logic. The key insight behind the construction of Beluga is that heavyweight upfront dissemination proofs (such as running an explicit Bracha broadcast) are *sufficient but not necessary* for robustness. Instead, it structures the push phase to carefully select the causal history of blocks based on scarcity signals (as opposed to simply selecting the first that arrives) and opportunistically leverages implicit dissemination evidence. The detection of missing blocks is performed through the reception of messages containing digests of past blocks unknown to the validator. In other words, the pull mechanism leverages the chaining and causal history of messages. Together, these mechanisms enable scarcity-aware pulls that provably bound recovery complexity while retaining optimal optimistic-path performance. In short, Beluga aims to preserve the optimistic high performance of current implementations while reintroducing the robustness guarantees of classic reliable broadcast.

Real world impact. We implement Beluga within Mysticeti, the consensus core of the Sui blockchain [1]. Our evaluation on a geo-distributed AWS testbed shows that Beluga maintains optimal optimistic-path latency and throughput while significantly improving performance under targeted attacks and adverse network conditions. Under attack scenarios, Beluga improves throughput by over 3× and reduces latency by 25× compared to baseline Mysticeti with its default block synchronizer (Sec. 6). We collaborated with the Sui team for over a year to integrate Beluga into the Sui blockchain. Beluga has been live since January 2025 with mainnet-v1.42.0, helping to secure over 5B USD in assets. Figure 1 shows the direct impact of Beluga: under network degradation attacks, Sui’s tail latency improved by 5x after deploying Beluga.

Contributions. We make the following contributions:

- (1) We formalize the block synchronizer interface and threat model, and derive baseline limitations of existing ad hoc mechanisms.
- (2) We design and analyze the *pull induction attack*, which increases latency by up to 50× and degrade throughput by up to 15× in existing systems.

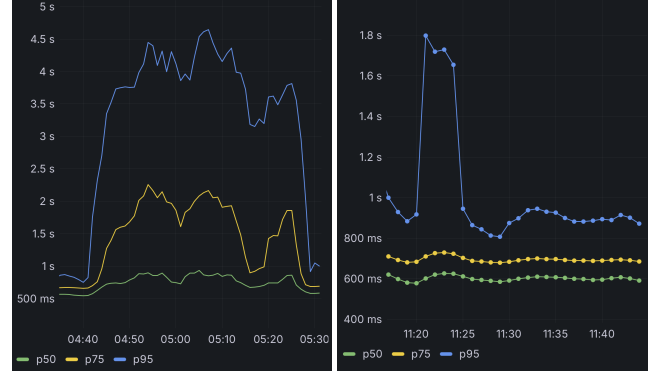


Figure 1: Latency comparison of the (production) Sui blockchain under network attacks before (left) and after (right) deploying Beluga.

- (3) We propose Beluga, a structured synchronizer with scarcity-aware detection and diversity-maximizing pull scheduling that preserves optimistic latency while bounding amplification.
- (4) We implement and integrate Beluga into Mysticeti/Sui, demonstrating no optimistic-path regression and significant under-attack throughput (3×) and latency (25×) improvements.

2 Problem Definition

Network model: We assume a set $\mathcal{V}$ of n validators (or parties; both are used interchangeably throughout this work) $\{v_1, \dots, v_n\}$ and a static adversary $\mathcal{A}$ that can corrupt up to $f < n/3$ of the parties arbitrarily, at any point. A party is *crashed* if it halts prematurely at some point during execution. Parties that deviate arbitrarily from the protocol are called *Byzantine* or *bad*. Parties that are never crashed or Byzantine are called *honest*. Parties are communicating over a partially synchronous network [22], in which there exists a special event called Global Stabilization Time (*GST*) and a known finite time bound Δ (we use δ to represent the actual network latency), such that any message sent by a party at time x is guaranteed to arrive by time $\Delta + \max\{GST, x\}$.

Threat model The adversary is computationally bounded. Pair-wise points of communication between any two honest parties are considered *reliable*, i.e., any honest message is *eventually* (after a finite, bounded number of steps) delivered. However, until *GST* the adversary controls the delivery of all messages in the network, with the only limitation that the messages must be eventually delivered. After *GST*, the network becomes synchronous, and messages are guaranteed to be delivered within Δ time after the time they are sent, potentially in an adversarially chosen order.

2.1 The Block Synchronizer Problem

We formalize the block synchronization problem here. We call it *block synchronizer*. As preliminaries, a block B contains the following basic data fields: round number r , block digest d , block creator *author*, connected parent blocks (in digest) *parents*, a list of transactions *payload*, and the creator’s *signature* on B .

DEFINITION 1 (BLOCK SYNCHRONIZER). *In the block synchronizer problem, a group of n validators $\mathcal{V}$ (of which up to f are Byzantine) collectively builds a structured, non-empty, and ever-growing set of*

blocks that are indexed by a monotonically increasing round number. Each validator v_i can call $\text{block_propose}_i(B, r)$ to push its block B in round r (where $B.r=r$) to the system. Each validator v_i outputs $\text{block_accept}_i(B.d)$ to accept B . The block B must include k blocks (from distinct validators) that $B.\text{author}$ has accepted in round $B.r-1$ as $B.\text{parents}$, where k is a parameter specifying how blocks are structured. Each validator v_i then outputs $\text{block_store}_i(B)$ to store B . The protocol should satisfy the following properties:

- **Round-Progression:** In each global round r of the system, at least $2f+1$ validators (not necessarily honest) invoke block_propose to create and disseminate their blocks. In other words, for each given $r \geq 0$, at least $2f+1$ produce blocks that have r as their round.
- **Round-Termination:** For each global round r of the system, every honest validator eventually outputs block_accept for the blocks produced in this round by least $2f+1$ validators. That is, for each given $r \geq 0$, each honest validator accepts block proposals, whose assigned round is r , from at least $2f+1$ different validators.
- **Block availability:** For some block B produced in round r , if an honest validator v_i outputs $\text{block_accept}_i(B.d)$, then v_i eventually outputs $\text{block_store}_i(B)$.
- **Causal availability:** For some block B produced in round r , if an honest validator v_i outputs $\text{block_accept}_i(B.d)$, then for every block $B' \in \text{causal}(B)$, v_i eventually outputs $\text{block_accept}_i(B'.d)$, where $\text{causal}(B)$ represents B 's causal history (i.e., all blocks for which there is a connection or path from B to them).

After comprehensively reviewing the existing relevant multi-proposer consensus protocols, we observe that they have some integrated block synchronization modules, each of which is composed of a push protocol and an optional pull protocol. For the sake of comparison, we define several performance-related metrics: (i) *push latency*, which is the network latency required to complete one round of block production, i.e., the interval between the creation of blocks in two consecutive rounds, (ii) *pull latency*, which is the network latency to fetch a block, and (iii) *pull complexity*, which is the communication complexity per node of fetching a block. Intuitively, push latency measures the protocol's data dissemination speed, and we consider push latency to be *optimal* if it equals δ throughout this paper, while pull latency and complexity indicate the overhead of block synchronization.

With the defined metrics, we consider that an ideal block synchronizer module/protocol must satisfy the following goals:

- (G1) **Optimal push:** The push latency is optimal under optimistic cases where the protocol progress is not stalled by the adversary and the network is synchronous.
- (G2) **Bounded amplification:** Under adverse cases, block retransmission and pull latency caused by the adversary are bounded when the network is synchronous. That is, the adversary cannot trigger infinite block retransmission and unbounded latency to synchronize blocks.

2.2 Existing Synchronizer Protocols

In this section, we explore the block synchroniser protocols in the existing works, where we model each class to the best of our abilities by both studying their research and implementations. The comparison is summarized in Table 1.

2.2.1 Multi-chain Certified Synchronizer Protocol. Many multi-chain BFT protocols, such as Autobahn [24] and Star [21], are built on the Multi-chain certified synchronizer protocol. Validators structure blocks into multiple parallel chains, where each block includes only one block from the same validator in the last round as parents (i.e., $k=1$). The Multi-chain certified synchronizer protocol consists of a weak quorum-based push protocol to push blocks and a deterministic pull protocol to fetch missing blocks.

Weak quorum-based push. For each round r , every validator v_i calls $\text{block_propose}_i(B, r)$ to push a round r block B into the system. Specifically, upon outputting block_accept for its own block in round $r-1$, v_i moves to round r and create a new block B with v_i 's round $r-1$ block in $B.\text{parents}$. Then, v_i employs a two-step Propose-Vote scheme to disseminate B . In the first step, v_i broadcasts B . In the second step, other validators respond to v_i with signatures on B and cache B . Note that validators do not output block_store at this point, since validators store a block only after it is certified. After that, v_i uses $f+1$ signatures to construct a *weak* certificate $WC(B.d)$ for B and outputs $\text{block_accept}_i(B.d)$ and $\text{block_store}_i(B)$. $WC(B.d)$ will be piggybacked into v_i 's round $r+1$ block B' . The other validator v_j receiving B' will output $\text{block_accept}_j(B.d)$ and output $\text{block_store}_j(B)$ if receiving B .

Deterministic pull. Validators in the Multi-chain certified synchronizer protocol perform the pull protocol independently from the push protocol. Specifically, when a validator v_i needs to pull a missing block B , v_i deterministically chooses a set of $f+1$ validators $\mathcal{V}_B$ in $WC(B.d)$ (i.e., validators who sign B) and sends a request message to $\mathcal{V}_B$. Since validators in $\mathcal{V}_B$ cache B , at least one honest validator can serve as the provider of B . Therefore, v_i must be able to receive B . v_i outputs $\text{block_store}_i(B)$ once $\text{block_accept}_i(B.d)$ succeeds and it receives B .

The Multi-chain certified synchronizer protocol achieves bounded amplification (G2), but not optimal optimistic-path push latency (G1). Specifically, it achieves a push latency of 2δ with the Propose-Vote scheme and a bounded pull latency of 2δ with the deterministic pull protocol. The pull complexity is $O(n)$.

2.2.2 DAG-based Certified Synchronizer Protocol. The certified DAG protocols, such as Bullshark [43] and Shoal++ [5], are built on a DAG-based certified synchronizer protocol. Validators organize blocks into a directed acyclic graph (DAG) using the quorum-based broadcast primitive. It consists of a consistent broadcast (CBC)-based push protocol to push certified blocks and a deterministic pull protocol to fetch missing blocks.

CBC-based push. For each round r , every validator v_i pushes a round r block B into the system by calling $\text{block_propose}_i(B, r)$. Specifically, upon outputting block_accept for blocks in round $r-1$ from at least $2f+1$ validators, v_i moves to round r and creates a new block B with *all* these round $r-1$ blocks as $B.\text{parents}$ (thus, $k=2f+1$). Then, v_i employs a three-step certificate scheme to disseminate B . In the first step, v_i broadcasts B . In the second step, other validators respond to v_i with signatures on B and cache B . Note that validators do not output block_store at this point, since validators can store a block in the DAG only after it is certified. Then, v_i uses these $2f+1$ signatures to construct a certificate $C(B.d)$. In the third step,

Table 1: Comparison of synchronizer protocols and their integrations to BFT consensus, after GST and the leader is honest

Consensus Protocols	Synchronizer Protocols	Optimistic Case			Adverse Case ⁽²⁾			Optimal Push?	Bounded Amplification?
		Push Latency	Consensus Latency ⁽¹⁾	Pull Complexity	Push Latency	Consensus Latency	Pull Complexity		
Star [21]	Multi-chain certified	2δ	5δ	$O(n)$	2δ	5δ	$O(n)$	✗	✓
	Beluga	δ	5δ	$O(1)$	$2\kappa\delta^{(3)}$	$4\kappa\delta+3\delta$	$O(n)$	✓	✓
Autobahn [24]	Multi-chain certified	2δ	7δ	$O(n)$	2δ	7δ	$O(n)$	✗	✓
	Beluga	δ	7δ	$O(1)$	$2\kappa\delta$	$4\kappa\delta+5\delta$	$O(n)$	✓	✓
Bullshark [43]	DAG-based certified	3δ	6δ	$O(n)$	5δ	10δ	$O(n)$	✗	✓
	Beluga	δ	4δ	$O(1)$	$2\kappa\delta$	$8\kappa\delta$	$O(n)$	✓	✓
Shoal++ [5]	DAG-based certified	3δ	4δ	$O(n)$	5δ	8δ	$O(n)$	✗	✓
	Beluga	δ	3δ	$O(1)$	$2\kappa\delta$	$6\kappa\delta$	$O(n)$	✓	✓
Sailfish [36]	DAG-based RBC	4δ	5δ	None	4δ	5δ	None	✗	✗
	Beluga	δ	3δ	$O(1)$	$2\kappa\delta$	$6\kappa\delta$	$O(n)$	✓	✓
Sailfish++ [37]	DAG-based RBC	2δ	3δ	None	3δ	4δ	None	✗	✗
	Beluga	δ	3δ	$O(1)$	$2\kappa\delta$	$6\kappa\delta$	$O(n)$	✓	✓
Mysticeti [6]	DAG-based uncertified	δ	3δ	$O(1)$	$\geq 3\delta$	$\geq 9\delta$	$O(1)$	✓	✗
	Beluga	δ	3δ	$O(1)$	$2\kappa\delta$	$6\kappa\delta$	$O(n)$	✓	✓

⁽¹⁾ Consensus latency specifies how long the protocol requires to reach a consensus on a leader block.

⁽²⁾ The adverse case is when the adversary can *successfully* stall protocol progress via pull-induction attacks (detailed in Sec. 3.1) or by deviating from optimistic termination conditions in [36]. The latency of Beluga under adverse cases is upper bound in theory and might not be reached in practice without assuming a powerful adversary who can arbitrarily reschedule message delivery.

⁽³⁾ $\kappa = 1 + \frac{f}{R_L}$, where f is the number of malicious validators, and R_L is a system parameter (see Sec. 4.2 for more details). By setting $R_L \gg f$, Beluga achieves $\kappa=1$.

v_i broadcasts $C(B.d)$. Moreover, when receiving $C(B'.d)$, v_i outputs $block_accept_i(B'.d)$ and output $block_store_i(B')$ if receiving B' .

Deterministic pull. The DAG-based certified synchronizer protocol employs a deterministic pull protocol as well. v_i requests a missing block from the set of validators $\mathcal{V}_B$ in $C(B.d)$. v_i outputs $block_store_i(B)$ after $block_accept_i(B.d)$ succeeds and receiving B .

The DAG-based certified synchronizer protocol ensures bounded amplification (G2), but not optimal push latency (G2). Specifically, it can achieve a push latency of 3δ with the CBC-based push protocol and a pull latency of 2δ with the deterministic pull protocol. The pull complexity is $O(n)$.

2.2.3 DAG-based RBC Synchronizer Protocol. Many recent DAG-based BFT consensus protocols, such as Sailfish [36] and its variant Sailfish++ [37], are built on a DAG-based RBC synchronizer protocol. Validators organize blocks into a DAG using the reliable broadcast (RBC) primitive. The RBC ensures that if an honest validator receives a block, all honest validators will eventually receive it (i.e., ensuring the totality property). Therefore, the pull protocol is *theoretically* not needed in the DAG-based RBC synchronizer protocol, and it only contains an RBC-based push protocol.¹

RBC-based push. For each round r , every validator v_i pushes a round r block B into the system by calling $block_propose_i(B, r)$. Specifically, upon outputting $block_accept$ for blocks in round $r-1$ from at least $2f+1$ validators, a validator v_i moves to round r and creates a new block B referencing *all* these round $r-1$ blocks as $B.parents$. v_i disseminates B using an RBC protocol [2, 9, 17, 37]. If v_i is honest, according to the RBC's Validity property, every other

honest validator v_j will eventually deliver B (i.e., v_j will receive B). In addition, when v_i delivers B' that is reliably broadcast by another validator v_j , v_i checks if it can accept B' by checking if it has outputted $block_accept_i(B''.d)$ for every block $B'' \in casual(B')$. If yes, v_i outputs $block_accept_i(B'.d)$ and $block_store_i(B')$ right after. If no, v_i will put B' into a pending list and update the list whenever it outputs $block_accept$ for a new block.

The DAG-based RBC synchronizer protocol fails to guarantee both optimal push latency (G1) and bounded amplification (G2). Specifically, implementing an upfront reliable broadcast requires honest validators to continually transmit messages to an unresponsive adversary, leading to unbounded retransmissions. In addition, the push latency depends on the RBC protocol used, but none of them are optimal (i.e., δ). For instance, Sailfish [36] employs the RBC protocol from [17], leading to a push latency of 4δ . Sailfish++ [37] adopts picky RBC protocol to achieve a push latency of 2δ (still not optimal) in optimistic cases (where at least $\lceil \frac{n+2f-2}{2} \rceil$ validators behave honestly). Sailfish++'s RBC protocol has a round latency of 3δ when optimistic cases do not hold.

2.2.4 DAG-based Uncertified Synchronizer Protocol. Mysticeti [6] consensus protocol uses a DAG-based uncertified synchronizer protocol. Validators structure blocks into a DAG, where each block includes at least $2f+1$ blocks from the last round as parents. The DAG-based uncertified synchronizer protocol consists of a best-effort broadcast (BEB)-based push protocol to push blocks and a random pull protocol to fetch missing blocks.

BEB-based push. For each round r , every validator v_i pushes a round r block B into the system by calling $block_propose_i(B, r)$. When receiving a block B' from other validators, v_i checks if it has outputted $block_accept_i(B''.d)$ for every block $B'' \in casual(B')$. If

¹ However, we find the implementations of the DAG-based RBC synchronizer protocol remain built on Bullshark's codebase, which still employs a pull mechanism [30, 31], likely due to the practical challenges of implementing upfront RBC (discussed in Sec. 1).

yes, v_i outputs $block_accept_i(B'.d)$ and $block_store_i(B')$ right after. If no, v_i uses the pull protocol to get *all* missing blocks. In essence, v_i must synchronize the whole causal history of B' before outputting $block_accept$. Upon outputting $block_accept$ for blocks in round $r-1$ from at least $2f+1$ validators, v_i moves to round r and creates a new block B with *all* these round $r-1$ blocks as $B.parents$. v_i broadcasts B in a best-effort way.

Random pull. To pull a missing block B in the DAG-based uncertified synchronizer protocol, the validator v_i randomly chooses a constant set of validators $\mathcal{V}_B \subseteq \mathcal{V}$ and sends a request message to $\mathcal{V}_B$. v_i repeatedly sends the request message to different sets of validators at intervals until receiving B . However, since B is not certified and v_i requests B from randomly selected validators, there is no guarantee that v_i will ever receive B . After getting B , v_i repeats the pull protocol to get all missing blocks of $causal(B)$. During the pull process, v_i outputs $block_accept_i(B.d)$ and $block_store_i(B)$ consecutively once it has synchronized B and all blocks in $causal(B)$.

The DAG-based uncertified protocol achieves optimal push latency (G1), but not bounded amplification (G2). Specifically, it achieves the round latency of δ with best-effort broadcast under optimistic cases. However, due to its random pull, it introduces uncertain pull latency and unbounded pull requests, thereby potentially unbounded push latency for every future round, despite having $O(1)$ pull complexity.

3 Pull Induction Attacks and Key Insights

Despite the rich design space proposed by prior work, none of them have taken a principled approach and achieved all ideal goals. This allows us to exploit their vulnerabilities through a new class of attacks we call pull induction attacks, which deliberately trigger unnecessary pulls to degrade their performance. This section sheds light on the pain points of existing protocols and provides several key insights that guide the design of Beluga.

3.1 Pull Induction Attacks

The goal of pull induction attacks is to induce honest validators to pull blocks from others, thereby increasing the round latency. To this end, the adversary selectively disseminates its blocks to only a subset of honest validators. Consequently, validators that do not receive the adversary's blocks are compelled to pull the missing blocks that are included by the protocol. Table 1 (adverse case column) presents a performance comparison of different block synchronizer protocols under pull induction attacks.

We give a concrete example of a pull induction attack against the DAG-based uncertified synchronizer protocol adopted by Mysticeti. We select Mysticeti as an example system since it is a state-of-the-art protocol widely adopted in industry [1, 44]. We leave the discussion of pull induction attacks against other synchronizer protocols in the full paper. In Figure 2, there are four validators v_1, v_2, v_3 , and v_4 , of which v_4 is the adversary. In round $r-1$, v_4 only disseminates its round $r-1$ block B_4^{r-1} to v_1 , making v_1 's round r block B_1^r reference B_4^{r-1} . When pushing round r blocks and receiving B_1^r , both v_2 and v_3 miss B_4^{r-1} , and thus, they invoke the random pull protocol to fetch B_4^{r-1} before accepting B_1^r and having enough (i.e., 3 with $n=4$ and $f=1$) accepted round r blocks to propose their round $r+1$

blocks. The latency of round r thus consists of δ for pushing round r blocks and at least 2δ for pulling B_4^{r-1} . Similarly, in round r , v_4 only disseminates its round r block B_4^r to v_2 , making v_2 's round $r+1$ block B_2^{r+1} reference B_4^r . This will induce both v_1 and v_3 to pull B_4^r before accepting B_2^{r+1} , thereby increasing the latency of round $r+1$ to at least 3δ . In round $r+1$, v_4 only disseminates its round $r+1$ block B_4^{r+1} to v_3 , which will induce both v_1 and v_2 to pull B_4^{r+1} before accepting B_3^{r+2} , thereby increasing the latency of round $r+2$ to at least 3δ . Recall that the consensus of Mysticeti requires three rounds of blocks [6, Algorithm 3]. As a result, the consensus latency of Mysticeti is at least 9δ under this pull induction attack.

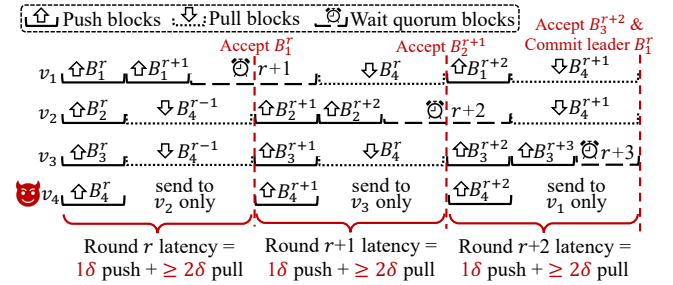


Figure 2: The pull induction attacks: the adversary v_4 selectively shares its blocks with honest validators. In round $r-1$, v_4 only shares its round $r-1$ block B_4^{r-1} with v_1 , making v_1 's round r block B_1^r reference B_4^{r-1} . This will induce v_2 and v_3 to trigger the pull protocol to fetch B_4^{r-1} to accept B_1^r , eventually increasing the latency of round r . Similarly, v_4 only shares its round r block with v_2 and shares its round $r+1$ block with v_3 . This will increase the latency of each round by at least one pull round-trip.

3.2 Key Insights

After reviewing existing block synchronizer protocols and their behavior under pull induction attacks, we have the following insights.

Key insight 1: DAG-based synchronizer protocols can accelerate the consensus latency, but at risk of being delayed by the adversary. In DAG-based synchronizer protocols, each block references at least $2f+1$ parents, and the connections between blocks can serve as proposal votes. This allows validators to complete the consensus during the formation of the DAG without extra communication by interpreting the DAG structure locally. As a result, the consensus latency of the protocols built on DAG-based synchronizer is soundly low under happy cases (e.g., 3δ in Mysticeti and 4δ in Sailfish). In contrast, the consensus protocols built on Multi-chain certified must rely on a dependent consensus process to order blocks, which introduces extra communication overhead. For instance, apart from 2δ round latency for pushing a round of blocks, Autobahn requires an additional 5δ network latency to reach a consensus on a leader block, leading to the consensus latency of 7δ .

However, forcing a block to reference $2f+1$ parents enables the adversary to delay the progress of the synchronizer protocol with the pull induction attacks. Specifically, when an honest validator has its block reference some adversary blocks that are not shared with other honest validators, these honest validators must pull the missing adversary blocks before accepting the honest validator's

block and collecting enough $2f+1$ accepted blocks to move to the next round. This increases the round latency by at least one pull round-trip. As a result, both round latency and consensus latency increase under the pull induction attacks.

Key insight 2: Block certificates allow performing the push and pull protocols separately. In the Multi-chain certified synchronizer protocol, validators push each block accompanied by a certificate containing a quorum of $\geq f+1$ signatures, thereby attesting the block and causal availability. In this case, validators can output *block_accept* for each certified block they receive without pulling any missing blocks in the block's causal history while still ensuring the Causal availability property. This enables the pull protocol to be performed in separation from the push protocol. We call this feature pulling blocks *off the push path*, as pulling blocks can be completed independently from the push protocol. In contrast, blocks are pulled *on the push path* if pulling missing blocks is necessary before validators accept them.

Supporting pulling blocks off the push path prevents Byzantine validators from proactively delaying the progress of the synchronizer protocol via pull induction attacks, since pulling missing blocks does not prevent validators from creating new blocks. However, the Multi-chain certified synchronizer adopts an explicit certificate mechanism, where certificates are created within at least one round-trip latency, leading to higher push and consensus latencies compared to the DAG-based uncertified synchronizer protocol.

Key insight 3: The pull protocol introduces a trade-off between round latency and communication complexity. In certified synchronizer protocols (e.g., DAG-based certified and Multi-chain certified), the pull is deterministic, where validators pull missing blocks from a specific set of validators (with the set size $O(n)$). This ensures validators can fetch the missing blocks within a constant round trip (i.e., 2δ with one for sending requests and the other for receiving blocks). However, this also introduces high communication complexity per validator (i.e., $O(n)$) since each validator might receive redundant blocks from others.

In contrast, the DAG-based uncertified synchronizer adopts a random pull protocol, where validators randomly pull the missing blocks from a small set of validators (with the set size $O(1)$). This reduces the communication complexity per validator to $O(1)$. However, such a random pull cannot ensure a constant round trip for synchronizing missing blocks. This is not a problem when the protocol is under happy cases, as pulling blocks does not impede progress. However, under adverse cases, the DAG-based uncertified synchronizer protocol requires at least 2δ in the pull protocol, leading to at least 3δ round latency and at least 9δ consensus latency.

4 The Beluga Protocol

4.1 Overview

Beluga is an efficient and robust DAG-based block synchronizer protocol composed of two key components: an AC-based optimistic push protocol (Sec. 4.2) and a hybrid pull protocol (Sec. 4.3) based on the novel idea of Implicit Proof-of-Availability (ImPoA).

AC-based optimistic push. Motivated by Insight 1, Beluga adopts a DAG structure and employs an optimistic push protocol, similar to the DAG-based uncertified synchronizer, to achieve optimal round

latency under happy cases (G1) and bounded retransmission (G2). Specifically, validators disseminate blocks using a best-effort broadcast, resulting in δ round latency. However, unlike the DAG-based synchronizer protocols, where validators arbitrarily reference parent blocks, Beluga introduces an *admission control (AC)* mechanism to filter out blocks based on the creators' behaviors. With AC, honest validators avoid referencing parent blocks created by suspected Byzantine validators—specifically, those that previously triggered them to invoke the pull protocol. This mechanism effectively safeguards Beluga against pull induction attacks.

ImPoA-based hybrid pull. Motivated by Insights 2 and 3, Beluga's pull protocol aims to separate pulling from pushing and balance pull complexity and latency. To this end, Beluga introduces an Implicit Proof-of-Availability (ImPoA) mechanism, which enables validators to identify blocks that can be safely accepted even if some of their ancestors are temporarily unavailable, thereby enabling validators to pull blocks off the push path. Based on ImPoA, we categorize pulling blocks into two types: *live blocks* and *bulk blocks*. Live blocks contribute to the quorum formation of the latest round but have unavailable ancestors; validators must accept these blocks to progress to the next round and propose new ones. In contrast, bulk blocks do not contribute to quorum formation in the current round. Leveraging this distinction, Beluga employs a hybrid pull strategy: live blocks are pulled deterministically to minimize the latency, while bulk blocks are pulled randomly to reduce pull complexity. This allows Beluga to achieve bounded pull latency (G2).

Beluga Workflow. In Beluga, each validator v_i pushes a block B to the system by invoking *block_propose_i*(B, r) every round r , which consists of parent selection and broadcast. Specifically, v_i becomes ready to push a block for round r after outputting *block_accept* for blocks in round $r-1$ from at least $2f+1$ distinct validators (including itself). v_i outputs *block_accept_i*($B'.d$) for a received block B' when it either has received all ancestors of B' , or for each parent, there exists an implicit certificate in v_i 's local view (i.e., the parent is referenced by $\geq f+1$ blocks; see Sec. 4.3.1). After that, v_i selects as parents a set of blocks that have been output via *block_accept*, subject to the admission control rules, and constructs a new block B for round r . B is then disseminated using a best-effort broadcast. Concurrently, v_i executes the ImPoA-based hybrid pull protocol to fetch missing blocks. v_i outputs *block_store_i*(B) to add B into the DAG once it has received B and output *block_accept_i*($B.d$).

Block structure. To capture validators' behaviors during the block pushing process and facilitate the pull process, Beluga augments the block structure with three additional fields.

- **Weak links:** The *weaklinks* field references blocks that a validator has received and accepted but not selected as parents. We therefore called *parents* as strong links, both are used interchangeably throughout the paper.
- **Watermark:** The *watermark* is an n -element array maintained by each validator, where the i -th entry implies the highest round number of block received from validator v_i .
- **Ancestors:** The *ancestors* is an n -element array storing, for each validator v_i , the highest round number of v_i 's blocks reachable from the current block.

4.2 AC-based Optimistic Push Protocol

Beluga’s push protocol specifies how validators create blocks and disseminate their created blocks to others. Block dissemination in Beluga is optimistic and relies on a best-effort broadcast; that is, validators disseminate blocks to all others without waiting for acknowledgments. Block creation is governed by an *Admission Control* (AC) module, which enforces rules that filter blocks according to the creators’ behaviors as quantified by a *reputation* mechanism. We detail the reputation mechanism and the AC module below, and defer the pseudo-code to Appendix E (Figure 8) due to page limit.

Reputation Mechanism. Each validator v_i maintains a local reputation table TR_i that records the reputations of all validators and is initialized with 0s. The reputation entry $TR_i[j]$ reflects the contribution of validator v_j to the block pushing process. Specifically,

- *Reputation Increase:* v_i increases v_j ’s reputation if it receives $2f+1$ blocks (denoted by $\mathcal{B}_r$) in round r that collectively indicate v_j ’s round $r-1$ block has been received. Formally, if for each block $B \in \mathcal{B}_r$, $B.\text{watermark}[j] = r-1$, then $TR_i[j]$ is incremented.
- *Reputation Decrease:* v_i decreases v_j ’s reputation whenever (i) it invokes the pull protocol to fetch a missing block created by v_j , or (ii) it receives pull requests for a v_j ’s block from at least $f+1$ distinct validators. A pull request serves as a *report* of v_j ’s delayed dissemination behavior. Collecting $f+1$ such reports constitutes a *blame* against v_j , indicating that at least one honest validator failed to receive v_j ’s block in a timely manner. Thus, v_i will decrease v_j ’s reputation with a blame.

Intuition behind the reputation mechanism. The reputation mechanism is designed to capture validators’ behaviors during the block pushing process. Specifically, if a validator v_i consistently pushes its blocks to all other validators, honest validators will frequently observe $2f+1$ blocks indicating that v_i ’s latest block is disseminated and received timely, thereby increasing v_i ’s reputation. In contrast, if v_i selectively pushes its blocks to only a subset of validators—performing a pull induction attack that forces others to invoke the pull protocol to retrieve its blocks—its reputation will decrease. Consequently, honest validators naturally maintain high reputations, whereas malicious validators that frequently launch such attacks accumulate low reputations.

In Beluga, the reputation increase is set by 1, while the reputation decrease is set by a large value R_L (e.g., $R_L=10,000$). This asymmetry minimizes the number of rounds that the adversary can delay and eventually enables Beluga to achieve decent round and consensus latencies. A detailed analysis is presented in Appendix C.

Admission Control. The AC module determines which blocks are selected as parents for newly created blocks based on their creators’ reputations. Upon advancing to round r and creating a new block, v_i first collects the latest blocks it has received from all validators with round numbers $\leq r-1$, denoted by the set $\mathcal{B}^{r-1}$. It then selects parent blocks from $\mathcal{B}^{r-1}$ through the following steps: (i) filter out any block in $\mathcal{B}^{r-1}$ that is not from round $r-1$ or is deemed unacceptable (i.e., v_i has not yet output *block_accept* for these blocks); (ii) from the remaining set, select the top $2f+1$ blocks in based on their creators’ reputations in TR_i . A block is considered *acceptable* if v_i has received all of its ancestors or can otherwise ensure its ancestors are available (see Sec. 4.3.1 for more details). This AC

mechanism ensures that honest validators avoid referencing blocks created by suspected Byzantine validators—specifically, those with low reputations due to previously triggering pull requests. By excluding such malicious blocks, an honest validator can create new blocks whose ancestors have already been received and accepted by all other honest validators, without invoking the pull protocol during the push process. Consequently, Beluga is inherently protected against pull induction attacks.

Apart from parents, a validator also references other acceptable blocks in $\mathcal{B}^{r-1}$ as *weaklinks* in the new block. The *weaklinks* serve as evidence of block availability to facilitate the pull process. We discuss it in Sec. 4.3.

Discussion. The AC-based optimistic push protocol faces a practical challenge when validators advance rounds. On the one hand, due to the asymmetric connectivity, high-reputation blocks may not be received first by honest validators. If validators immediately reference the first received (potentially low-reputation) blocks as parents, the protocol may violate its intended guarantees. On the other hand, during asynchrony, reputation updates can be temporarily inaccurate, causing honest validators to appear lower-ranked than malicious ones. If validators wait for high-reputation blocks, malicious validators can delay progress arbitrarily or even stall the protocol by withholding their blocks.

Beluga addresses this challenge by introducing a reputation threshold R_t , a per-round timeout T_{rd} , and periodic reputation resets. Specifically, a validator v_i advances to round r if it either: (i) receives $2f+1$ blocks from round $r-1$ whose creators have reputations above a threshold $R_t = R_{2f+1} - R_L$, where R_{2f+1} is the $2f+1$ -th highest reputation in TR_i ; or (ii) the per-round timeout T_{rd} expires, which is set to 4Δ to ensure all honest blocks are received (Lemma 1). In brief, after GST and a reputation reset, v_i advances to new rounds using either honest blocks or blocks that do not hinder protocol progress.

4.3 ImPoA-based Hybrid Pull Protocol

Beluga’s pull protocol defines how validators fetch missing blocks when they cannot be accepted during the push process. Beluga’s pull protocol is called the implicit proof-of-availability (ImPoA)-based hybrid pull protocol. As illustrated in Figure 3, it comprises two components: an ImPoA-based pull mechanism and a hybrid pull strategy.

4.3.1 ImPoA-based Pull Mechanism. The ImPoA-based pull mechanism enables validators to pull certain blocks off the push path once their availability can be proven. However, unlike the existing certified synchronizer protocols (such as Multi-chain certified) that create *explicit* block certificates at the cost of additional communication overhead during the push process, Beluga constructs *implicit* block certificates to achieve the same availability guarantees by locally interpreting block patterns.

Implicit PoA. In Beluga, a block B is identified as *implicitly available* if it is referenced (via strong or weak links) by at least $f+1$ blocks from the subsequent rounds. Note that a validator references B only if it (i) receives B , and (ii) can verify the availability of B ’s causal history (lines 4-5, Figure 8). Consequently, these $f+1$ referencing blocks collectively form an implicit proof-of-availability

(PoA) for B , implying that at least one honest validator can attest to B 's causal availability. For instance, in Figure 3(a), the missing block B_1^{r+1} is identified implicitly available as it was referenced by two received blocks B_2^{r+2} and B_4^{r+2} .

Live and Bulk modules. With ImPoA, validators can safely accept blocks even when parts of their causal histories are missing, thereby allowing missing blocks to be pulled off the push path. To accommodate the mechanism, Beluga employs two pull synchronizer modules: (i) *bulk synchronizer module*, which retrieves missing blocks off the push path, and (ii) *live synchronizer module*, which retrieves missing blocks on the push path.

Mechanism specification. We now describe the workflow of our pull mechanism. Assume a validator v_i is currently proceeding in round r . Upon receiving a block B containing missing parents (i.e., B has strong links to some blocks that v_i has not accepted) during the push process, v_i determines which pull synchronizer module will be used to pull B 's missing ancestors based on B 's round number. We denote $\mathcal{B}_{bk}$ the block set in the bulk synchronizer module and $\mathcal{B}_{lv}$ the block set in the live synchronizer module.

If $B.r < r$, it means that v_i receives an old-round block that will not be included in v_i 's threshold clock (i.e., block quorum for round r). The pull synchronization of B will not affect the push protocol, and thus B could be synchronized off the push path. v_i then transmits B to the bulk synchronizer module and includes B into $\mathcal{B}_{bk}$.

If $B.r \geq r$, it means that B is a live block from v_i 's local view. However, instead of transmitting B to the live synchronizer module immediately, v_i first checks whether B can be proven available. Specifically, if every parent of B has an implicit PoA from the live block set $\mathcal{B}_{lv}$ or has been *block_accept*, then v_i ensures B is available. v_i then transmits B to the bulk synchronizer, outputs *block_accept* and *block_store* for B , and include B into the bulk block set $\mathcal{B}_{bk}$. If, otherwise, B is not proven available; v_i then transmits B to the live synchronizer module and includes B into $\mathcal{B}_{lv}$. For example, in Figure 3(a), v_4 is proceeding in round $r+2$ and transmits B_2^{r+2} and B_4^{r+2} to the bulk synchronizer module even though their common parent B_1^{r+1} was missing, since B_1^{r+1} has an implicit PoA. In contrast, B_3^{r+2} is transmitted to the live synchronizer module.

Moreover, v_i can dynamically transmit blocks from $\mathcal{B}_{lv}$ to $\mathcal{B}_{bk}$ according to the following conditions:

- (1) Upon advancing to a new round $r' > r$, v_i transmits any $B' \in \mathcal{B}_{lv}$ to $\mathcal{B}_{bk}$ if $B'.r < r'$.
- (2) Upon including a new block into $\mathcal{B}_{lv}$, v_i checks whether any block $B' \in \mathcal{B}_{lv}$ is proven available. If yes, v_i transmits such available B' from $\mathcal{B}_{lv}$ to $\mathcal{B}_{bk}$.

Identifying missing blocks. By utilizing the ImPoA-based pull mechanism, Beluga classifies blocks with missing ancestors into two categories: *Live blocks* (i.e., blocks in $\mathcal{B}_{lv}$) and *Bulk blocks* (i.e., blocks in $\mathcal{B}_{bk}$). Each validator v_i can then locally identify missing blocks from $\mathcal{B}_{lv}$ and $\mathcal{B}_{bk}$ by checking their *ancestors*. However, since blocks in the DAG are well-connected, $\mathcal{B}_{lv}$ and $\mathcal{B}_{bk}$ might involve overlapped missing ancestors. For instance, in Figure 3, the missing block B_1^{r+1} is the common ancestor of both the live blocks and the bulk blocks. As a result, v_i might pull the same missing blocks in both the live and bulk synchronizer modules redundantly.

To avoid redundant synchronization, v_i traces the last accepted blocks from all validators and combines them with blocks' ancestors to identify missing blocks for distinct synchronizer modules. Since each block must reference the previous block proposed by the same validator [6], the last accepted block B_j^r from v_j in round r indicates that all blocks from v_j with a round $r' \leq r$ are proven available and can be accepted by v_i . Moreover, if a validator accepts a block, it means that the block's causal history is available and can be accepted. As a result, with the information, v_i can identify the missing blocks that are *only* required by the live blocks.

As shown in Figure 3(b), the last accepted blocks and their *ancestors* information imply that v_4 (i) will be able to accept $r+1$ block B_1^{r+1} from v_1 , and (ii) has accepted round $r+2$ block B_2^{r+2} from v_2 , round $r-1$ block B_3^{r-1} from v_3 , and round $r+2$ block B_4^{r+2} from itself. When processing the pending block B_3^{r+2} (which references v_3 's round $r+1$ block) in the live synchronizer module, v_4 can identify B_3^r and B_3^{r+1} are only required by the live blocks (i.e., B_3^{r+2}) but B_1^{r+1} does not. As a result, the live synchronizer module only requests the missing blocks B_3^r and B_3^{r+1} , while the bulk synchronizer module requests the missing block B_1^{r+1} .

4.3.2 Hybrid Pull Strategy. After identifying the missing blocks that a validator v_i needs to fetch, v_i deploys a hybrid pull strategy to balance message and round complexities, as shown in Figure 3(c).

Pulling blocks in the live synchronizer. Blocks $\mathcal{B}_{lv}$ in the live synchronizer module are time-sensitive, as their missing blocks can block the push process. Thus, the live synchronizer module adopts a *deterministic pull strategy* to minimize the latency. Specifically, v_i sends the pull requests for all missing blocks specified in $\mathcal{B}_{lv}$ to all validators. Such a pull strategy might pull redundant blocks. However, it guarantees that v_i can receive missing blocks as long as they are *block_accepted* by one honest validator, and process live blocks within a round-trip delay (i.e., 2δ).

Pulling blocks in the bulk synchronizer. Pulling blocks in the bulk synchronizer does not block the push process, and thus, is not sensitive to latency. Consequently, the bulk synchronizer module adopts a *randomized rotating pull strategy* to minimize the pull complexity. Specifically, for each missing block specified in $\mathcal{B}_{bk}$, v_i randomly chooses one validator to send the pull request. If v_i does not receive the requested missing block within some predefined time (say Δ_{bk}), it randomly chooses another validator who has not been requested by v_i to send the pull request.

After getting missing blocks, v_i outputs *block_accept* and *block_store* for each of them if v_i hasn't done it before.

4.4 Building BFT Consensus on Beluga

Beluga is inspired by modern multi-proposer BFT protocols and can serve as a modular block synchronization layer for a broad class of partially synchronous BFT consensus protocols. As illustrated in Figure 4, Beluga interfaces with a generic state machine replication system by supplying accepted blocks (i.e., those that have been output via *block_accept*) to the consensus layer for ordering and supplying available blocks (i.e., those that have been output via *block_store*) to the execution layer for execution after these blocks

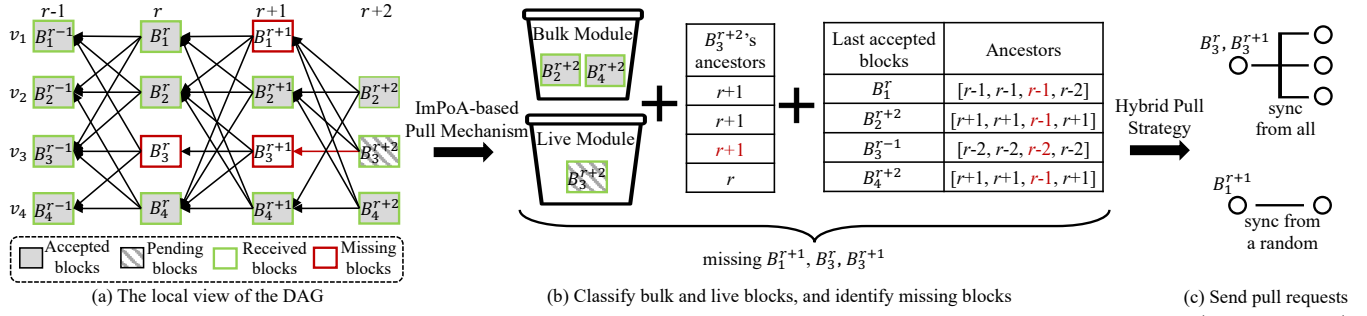


Figure 3: The ImPoA-based hybrid pull protocol for v_4 : (a) v_4 is proceeding in round $r+2$ but misses blocks B_1^{r+1} , B_3^r , and B_3^{r+1} . B_1^{r+1} is identified as implicitly available as $\{B_2^{r+2}, B_4^{r+2}\}$ reference it. As a result, v_4 accepts B_2^{r+2} and B_4^{r+2} even though their parent B_1^{r+1} is not received. (b) v_4 transmits blocks that reference missing ones to the bulk and live synchronizer modules via the ImPoA-based pull mechanism. With the hints, v_4 identifies the missing blocks it needs to fetch. (c) v_4 fetches blocks via a hybrid pull strategy, balancing pull latency and complexity.

are ordered. The key enabler of the generality is that the DAG structure constructed by Beluga induces structural *patterns* on blocks which can be used by different consensus protocols to implement their ordering logic.

Block patterns. We define two patterns that arise naturally in Beluga. A block forms an *availability pattern* if it is referenced by more than f blocks produced by distinct validators. A block forms a *certificate pattern* if it is referenced as parents by more than $2f$ blocks from distinct validators. Intuitively, the availability pattern ensures that the block and its causal history can be retrieved by honest validators; while the certification pattern implies uniqueness: for any validator and round, at most one block can become certified. A block that forms an availability pattern (resp., certificate pattern) is called an *available block* (resp., *certified block*). Both patterns involve two push rounds to generate. Moreover, we say that a block *votes for* a certified block B_C if it references at least $2f+1$ blocks as parents that themselves reference B_C .

Applying consensus on Beluga. These patterns enable BFT protocols to operate over the DAG constructed by Beluga while applying their own ordering rules. We illustrate this integration using the protocols listed in Table 1, and summarize the resulting performance in Table 1.

For multi-chain protocols such as Dashing [21] and Autobahn [24], validators execute separate consensus instances to ensure non-equivocation and thus only rely on Beluga to ensure block availability. They use availability patterns when performing separate consensus instances. Specifically, a leader validator uses its most recent block that forms an availability pattern as a proposal to coordinate a consensus instance. This design achieves improved push latency in optimistic conditions, at the cost of a modest increase in consensus latency under adverse conditions.

DAG-based protocols perform consensus by locally interpreting the DAG structure. When instantiated over Beluga, they rely on certificate patterns to ensure non-equivocation. In these protocols, certain blocks are designated as leader blocks to drive consensus progress. During the push phase, Beluga’s admission control prioritizes the inclusion of leader blocks and their supporting blocks from validators with benign reputations. Validators then apply protocol-specific rules to order blocks.

Protocols such as Mysticeti [6] and Cordial Miners [26] employ the same DAG structure as Beluga (i.e., constructing a DAG using a best-effort broadcast approach but distinct admission control and pull mechanisms). Therefore, their ordering rules can be applied to Beluga directly to derive a BFT consensus protocol by *waiting and including leader blocks as parents* when performing the push protocol, and by committing the leader blocks that form certification patterns. These consensus protocols achieve the same optimal push and consensus latencies in optimistic scenarios when instantiated over Beluga, while offering greater robustness under adverse conditions. Appendix D presents formal security proofs.

Protocols such as Bullshark [43] and successor Shoal [42] commit a leader certified block once it receives sufficient votes from subsequent certified blocks. When executed over Beluga, this process requires two rounds of certified blocks, resulting in a consensus latency of 4δ in optimistic cases and 8δ in adverse cases.

Protocols such as Shoal++ [5], Sailfish [36], and Sailfish++ [37] commit a leader certified block using one round of certified blocks and the first messages of the next round. A leader certified block is directly committed once at least $2f+1$ first messages of the next-round certified blocks vote for it. In Beluga, these first messages correspond to individual blocks, allowing commitment with three push rounds. This yields a consensus latency of 3δ in optimistic cases and 6δ in adverse cases.

Garbage collection. Beluga naturally reuses the garbage collection mechanism introduced by Narwhal [16]; this component allows the protocol to clean up any partially disseminated messages that were not promptly committed, preventing unbounded storage and memory growth. Notably, this module is employed in most implementations, both in DAG-based and linear BFT protocols [23, 29, 41].

5 Protocol Analysis

We prove Beluga satisfies the properties defined in Definition 1 (Sec. 2.1). Appendix C shows the performance of Beluga both in the happy case and under adversarial settings.

LEMMA 1. *After GST, all honest validators will enter the same round within 3Δ .*

PROOF. W.l.o.g, assume round r is the last round entered by honest validators before GST. After GST, the message delay between

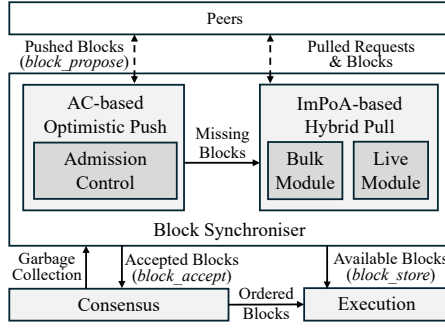


Figure 4: Beluga can be integrated into a generic BFT consensus protocol. By applying the consensus rules on the blocks produced by Beluga, validators derive a consistent order for blocks. Beluga guarantees the availability of ordered blocks for the state machine replication (SMR) execution.

honest validators is bounded by Δ . Thus, all honest validators must receive at least one round r block B_i^r from honest validator v_i by time $GST + \Delta$. Every honest validator can synchronize all B_i^r 's parent blocks and missing ancestors via the pull protocol within 2Δ . Consequently, all honest validator can accept at least $2f+1$ round $r-1$ blocks and enter round r by time $GST + 3\Delta$. $\square$

LEMMA 2. *After GST, if an honest validator v_i enters round r at time t_r , and all honest validators have created and disseminated their round r blocks by time t_r , then all honest validators will be able to enter round $r+1$ by time $t_r + 3\Delta$.*

PROOF. According to this lemma's condition, t_r is the time when the slowest honest validator creates and disseminates its round r block. By time $t_r + \Delta$, all honest validators will have received the round r blocks from all honest validators. Even though some honest validators might need to synchronize missing ancestors to accept some round r blocks, they can accept these blocks via our pull protocol within the time bound of 2Δ . As a result, all honest validators can accept at least $2f+1$ round r blocks by time $t_r + 3\Delta$ and enter their round $r+1$ blocks by time $t_r + 3\Delta$. $\square$

THEOREM 1 (BLOCK AVAILABILITY). *Beluga satisfies block availability. If an honest validator v_i outputs $block_accept_i(B.d)$ for some block B produced r , then v_i eventually outputs $block_store_i(B)$*

PROOF. In Beluga, an honest validator v_i outputs $block_accept_i(B.d)$ for some block B in round r when v_i received B . As a result, v_i must have stored B and output $block_store_i(B)$. $\square$

THEOREM 2 (CAUSAL AVAILABILITY). *Beluga satisfies causal availability. If an honest validator v_i outputs $block_accept_i(B.d)$ for some block B , then for every block $B' \in causal(B)$, v_i eventually outputs $block_accept_i(B'.d)$, where $causal(B)$ represents B 's causal history.*

PROOF. In Beluga, an honest validator v_i outputs $block_accept_i(B.d)$ for some block B in round r when v_i received B and ensures all its parents in round $r-1$ are available—that is, v_i has either outputted $block_accept$ for the parent blocks or observed they are referenced by at least $f+1$ subsequent blocks (Sec. 4.3). In the latter case where the parent blocks (denoted by a set $\mathcal{B}^{r-1}$) are referenced by at least $f+1$ subsequent blocks, according to Beluga's push protocol,

the creators of these $f+1$ subsequent blocks must have outputted $block_accept$ for $\mathcal{B}^{r-1}$. This means that at least one honest validator v_j has stored $\mathcal{B}^{r-1}$ and ensured all parents of $\mathcal{B}^{r-1}$ in round $r-2$ are available. As a result, v_j must have either outputted $block_accept$ for the parent blocks of $\mathcal{B}^{r-1}$ in round $r-2$ or observed they are referenced by at least $f+1$ subsequent blocks. By induction, we can see that for every block $B' \in causal(B)$, there exists at least one honest validator that has stored B' and ensured all its parents are available. As a result, v_i can eventually receive B' from this honest validator via the pull protocol and output $block_accept_i(B'.d)$. $\square$

THEOREM 3 (ROUND-PROGRESSION). *Beluga satisfies round-progression. For each round $r \geq 0$, at least $2f+1$ validators will create and disseminate their round r blocks.*

PROOF. For the genesis round 0, all validators will create and disseminate their round 0 blocks. Thus, the lemma holds for round 0. In addition, according to Lemma 1, all honest validators can enter the same round (w.l.o.g. at round r) within 3Δ after GST. As a result, all (i.e., at least $2f+1$) honest validators must be able to create and disseminate their round r blocks by time $GST + 3\Delta$. Thanks to Beluga's pull protocol, every honest validator can accept at least $2f+1$ round r blocks and create its round $r+1$ block. By induction, we can see that for any future round $r' > r$, at least $2f+1$ validators will create and disseminate their round r' blocks. Moreover, for any round $1 \leq r'' \leq r$, since Beluga's push protocol requires validators to reference at least $2f+1$ parent blocks from the previous round when creating their round r'' blocks, at least $2f+1$ validators must have created and disseminated their round $r'' - 1$ blocks. By induction, we can see that for any previous round $1 \leq r'' \leq r$, at least $2f+1$ validators must have created and disseminated their round r'' blocks. The proof is done. $\square$

THEOREM 4 (ROUND-TERMINATION). *Beluga satisfies round-termination. For each round $r \geq 0$, each honest validator accepts block proposals, whose assigned round r , from at least $2f+1$ different validators.*

PROOF. For the genesis round 0, since all honest validators create their round 0 blocks, and round 0 blocks do not reference any blocks, each honest validator can accept at least $2f+1$ round 0 blocks. The lemma holds for round 0. In addition, according to Theorem 3, for each round $r \geq 1$, at least $2f+1$ validators will create and disseminate their round r blocks. Each round r blocks consist of at least $f+1$ blocks created by honest validators. For these $f+1$ honest validators, according to Beluga's push protocol, they must output $block_accept$ to accept at least $2f+1$ round $r-1$ blocks. According to Theorem 1 and Theorem 2, these round $r-1$ blocks and their causal histories are available to all honest validators. As a result, each honest validator can accept at least $2f+1$ round $r-1$ blocks. By induction, we can see that for any round $r \geq 1$, each honest validator can accept at least $2f+1$ round r blocks. $\square$

6 Experimental Evaluation

We evaluate the throughput and latency of Mysticeti [6] equipped with Beluga through experiments conducted on AWS on a geo-distributed testbed. Appendices F and G describe our implementation and experimental setup in detail. We then show its improvements over the baseline Mysticeti implemented with the default

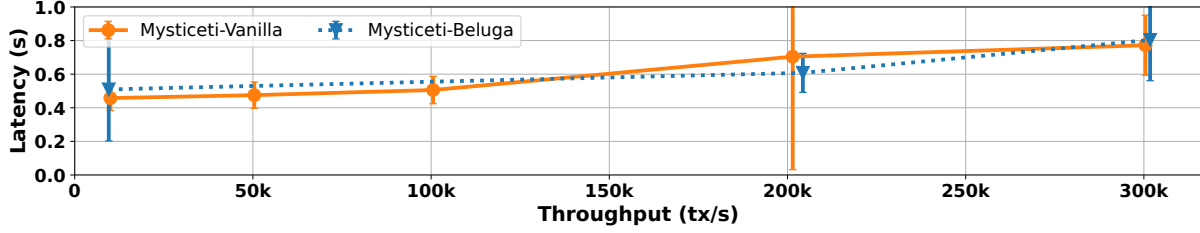


Figure 5: Comparative throughput-latency performance of Mysticeti equipped with Beluga and with the baseline push-pull block synchronizer. WAN measurements with 50 validators, no faults, and 512B transaction size.

push-pull block synchronizer module (Sec. 2). We chose to implement Beluga within Mysticeti because (1) it is a state-of-the-art DAG-based BFT consensus protocol deployed in production in multiple blockchains [1, 44], providing real-world impact, (2) its open-source codebase [27] is well documented and modular, facilitating implementation, and (3) it builds upon an uncertified DAG, making it particularly vulnerable to the attacks described in Sec. 3.1. These properties make Mysticeti an ideal candidate to demonstrate the effectiveness of Beluga.

Our evaluation demonstrates the following claims:

- C1** Beluga introduces no noticeable performance overhead when the protocol runs in ideal conditions (no Byzantine parties and synchronous network)
- C2** Beluga drastically improves both latency and throughput in the presence of asynchronous network conditions and the attack presented in Sec. 3.1.

Evaluating the performance of BFT protocols in the presence of generic Byzantine faults is an open research question [7], and state-of-the-art evidence relies on formal proofs.

6.1 Benchmarks in Ideal Conditions

Figure 5 depicts the performance of Mysticeti equipped with both Beluga and the baseline block synchronizer operating through a best-effort push protocol followed by a random pull (described in Sec. 2) running with 50 honest validators. As expected, the performance of the baseline Mysticeti is similar to Mysticeti equipped with Beluga. Both systems exhibit a stable throughput up to around 100,000 tx/s while maintaining a latency of around 0.5s, and both scale easily to 300,000 tx/s (512-byte transactions) while maintaining sub-second latency (0.8s). We did not push throughput further for cost reasons, but both systems appear to use less than 10% of their CPU at that point, indicating that they could likely handle even higher loads. This performance similarity is due to Beluga’s lightweight requirements during the happy path: when all parties are honest and the network is synchronous, Beluga imposes minimal constraints on parent block selection and thus operates similarly to the baseline push-pull synchronizer. This confirms Item **C1** that Beluga does not introduce any significant overhead when the network is synchronous and all parties are honest.

6.2 Benchmarks under Attack

Figure 6 depicts the performance of Mysticeti with both Beluga and the baseline block synchronizer in a 10-validator deployment with 1 or 3 faults (we limit this benchmark to 10 validators for cost reasons). The Byzantine validators perform the pull induction

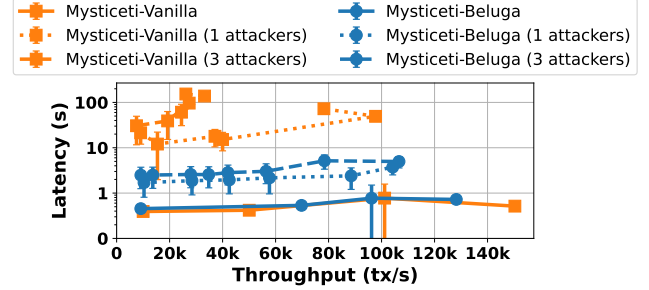


Figure 6: Comparative throughput-latency performance of Mysticeti equipped with Beluga and with the baseline push-pull block synchronizer. WAN measurements with 10 validators, zero, one, and three faults, and 512B transaction size. (Note the log scale on latency.)

attack described in Sec. 3.1, creating conditions that can also arise from severe network asynchrony. The baseline Mysticeti suffers severe throughput degradation and dramatic latency increases. For reference, the graph includes the no-fault performance with 10 validators to illustrate the performance gap under attack. With three Byzantine faults, baseline Mysticeti’s throughput drops by over 15x and its latency increases to over 50 seconds, compared to 0.5 seconds in the fault-free case. In contrast, Mysticeti equipped with Beluga maintains substantially higher throughput: Beluga rapidly detects and prioritizes inclusion of blocks from reliable validators in the DAG. The throughput reduction (30%) is primarily attributable to the loss of faulty validator capacity. Latency increases by approximately 4x to around 2 seconds, which is expected given the need to wait for additional blocks before proceeding to the next round and to recover from attack-induced asynchrony. This represents a substantial improvement over baseline Mysticeti: over 3x higher throughput and 25x lower latency with three Byzantine faults. These results confirm Item **C2** that Beluga significantly improves both latency and throughput under asynchronous network conditions and pull induction attacks.

7 From paper to Mainnet

The motivation for developing Beluga arose in March 2024 following a testnet incident on the Sui blockchain, triggered by a fundamental change in the protocol’s networking stack. Some validators misconfigured their machines, resulting in a situation where they could receive blocks from other validators but were unable to propagate their own. Consequently, these validators accumulated a large number of blocks that remained unshared with the network.

In one extreme case, a single validator locally created approximately 750,000 blocks that had not been disseminated. Once the validator corrected their configuration and broadcast all these blocks simultaneously, the network experienced a stall: fast validators began including these blocks in their DAGs, and once $f + 1$ of them did so, the rest of the network had to synchronize as well, causing a substantial performance degradation. Specifically, all 750,000 blocks were committed within one minute, triggering an overwhelming number of pull requests.

This incident revealed the root cause in the block synchronizer component. Its admission control module (see Figure 4) lacked sufficient filtering for poorly performing validators, allowing the most recent hoarded blocks to enter local DAGs. Moreover, the original pull protocol exacerbated the problem by indiscriminately pulling from both poorly performing and random validators, resulting in high synchronization latency. These observations underscored the need for a high-quality DAG that minimizes synchronization from underperforming validators.

We collaborated with the Sui team to experiment with various heuristics inspired by leader scoring [48], but initially, excluding ancestors indiscriminately disrupted block propagation even under benign conditions. This motivated the introduction of optional dependencies (Sec. 4). Subsequently, they embarked on a multi-month effort to collect detailed consensus metrics and evaluate different strategies for scoring validators. This process ultimately led to the design presented in Section 4. After nearly a year of testing and tuning, Beluga was deployed on Sui mainnet version mainnet-v1.42.0 in January 2025.

Figure 1 (Sec. 1) shows a production network replicating the Sui mainnet with all 135 validators, in their respective geo-location and with distribution stake, sustaining a constant load of 6,000 transactions per second. The figure compares the original Mysticieti protocol run within the Sui mainnet before Beluga deployment, and the improved version with Beluga. The results demonstrate that Beluga effectively mitigates the latency spikes previously observed in the network under attack, resulting in a stable and predictable transaction commit latency. Beluga effectively prevents poorly performing validators from adversely impacting the overall network performance, resulting in a 5x reduction in the 95th percentile, a 2x reduction in the 75th percentile, and a 25% reduction in the 50th percentile latency.

Figure 7 shows the rate at which blocks from an intentionally slow validator (green line) appear in the committed DAG under the same production setup after deploying Beluga. The validator's contribution to the committed DAG falls to zero within roughly 1-2 minutes as its score drops and the admission-control module at other validators discards its blocks. It remains zero throughout the attack, and then returns to its original rate within 3 minutes of recovering. Notice how other validators remain largely unaffected (other colored lines on the figure). This result demonstrates that Beluga rapidly identifies and deprioritizes poorly performing validators, preventing them from degrading network performance during misbehavior, and then promptly restores their ability to participate once they return to normal behavior. Thus, recovering validators rejoin the consensus process without long-term penalties.

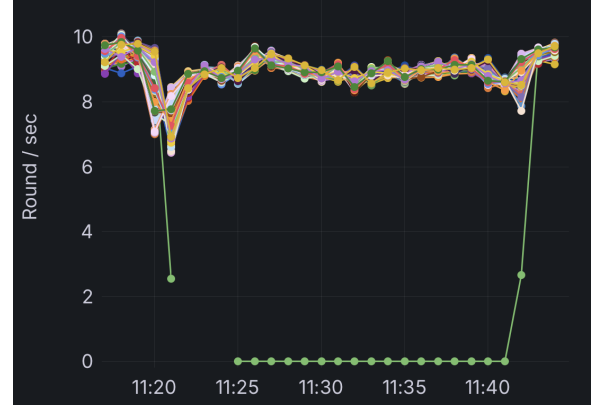


Figure 7: Block proposal rate of an intentionally slow validator in a Sui mainnet reproduction (135 validators) after deploying Beluga. Each line represents the proposal rate of a validator. Proposal rate quickly drops to zero during the attack and recovers shortly after performance is restored.

8 Related Work

A largely unexplored dimension of the consensus literature concerns the design of synchronization primitives themselves. While many protocols assume a specific synchrony model, prior work, to our knowledge, does not isolate, specify, or evaluate such primitives as first-class, modular components of consensus stacks. We address this gap by defining and implementing synchronization abstractions that compose cleanly with existing consensus protocols.

One key source of inspiration is the Pacemaker [8] of leader-based protocols. The pacemaker separates view-change timing, leader rotation, and timeout management from the safety-critical core, which creates clean interfaces and lets liveness mechanisms evolve independently. Our synchronizer plays a similar role: it fits into the modular blockchain architecture [15], defines isolated processes that deliver high-performance support to the safety-critical total-ordering algorithm, and lets engineers separate concerns.

A synchronizer protocol can be viewed as a decomposed form of reliable broadcast [32], one of the most studied problems in distributed computing. Unlike AVID [3, 12] protocols, which focus on communication complexity in asynchronous settings, or optimistic reliable broadcast [37], which emphasizes latency under a low number of faults, our design examines performance after GST in partially synchronous or synchronous networks. This approach provides a more practical perspective and has already enabled significant performance improvements in Sui, a top-20 blockchain.

In permissionless networks, GossipSub [49] addresses related concerns at the gossip overlay layer. Like Beluga, it decomposes message propagation into an eager push phase (via a structured local mesh) followed by a lazy pull phase (gossip-driven requests for missing messages), and it incorporates a peer scoring function that penalizes misbehaving nodes to maintain mesh quality. However, GossipSub targets open, Sybil-prone environments (Filecoin and Ethereum 2.0) and operates purely at the network transport layer, without awareness of causal block structure or consensus semantics. In contrast, Beluga is a consensus-aware module that exploits DAG structure for scarcity detection and provides provable reliable broadcast guarantees.

Acknowledgments

This paper was edited for grammar using ChatGPT.

References

- [1] 2024. Sui. <https://sui.io/>.
- [2] Ittai Abraham, Kartik Nayak, Ling Ren, and Zhuolun Xiang. 2021. Good-case latency of byzantine broadcast: A complete categorization. In *Proceedings of the 2021 ACM Symposium on Principles of Distributed Computing*. 331–341.
- [3] Nicolas Alhaddad, Sourav Das, Sisi Duan, Ling Ren, Mayank Varia, Zhuolun Xiang, and Haibin Zhang. 2022. Asynchronous verifiable information dispersal with near-optimal communication. *Cryptology ePrint Archive* (2022).
- [4] Paulo Sérgio Almeida and Ehud Shapiro. 2025. The Blocklace: A Byzantine-repelling and Universal Conflict-free Replicated Data Type. arXiv:2402.08068 [cs.DC] <https://arxiv.org/abs/2402.08068>
- [5] Balaji Arun, Zekun Li, Florian Suri-Payer, Sourav Das, and Alexander Spiegelman. 2024. Shoal++: High throughput dag bft can be fast! *arXiv preprint arXiv:2405.20488* (2024).
- [6] Kushal Babel, Andrey Chursin, George Danezis, Anastasios Kichidis, Lefteris Kokoris-Kogias, Arun Koshy, Alberto Sonnino, and Mingwei Tian. 2025. MYSTICETI: Reaching the Latency Limits with Uncertified DAGs. In *Network and Distributed Systems Security Symposium (NDSS)*.
- [7] Shehar Bano, Alberto Sonnino, Andrey Chursin, Dmitri Perelman, Zekun Li, Avery Ching, and Dahlia Malkhi. 2021. Twins: Bft systems made robust. In *ACM PODC*.
- [8] Mathieu Baudet, Avery Ching, Andrey Chursin, George Danezis, François Garillot, Zekun Li, Dahlia Malkhi, Oded Naor, Dmitri Perelman, and Alberto Sonnino. 2019. State machine replication in the libra blockchain. *The Libra Assn., Tech. Rep* 7 (2019).
- [9] Gabriel Bracha. 1987. Asynchronous Byzantine agreement protocols. *Information and computation* 75, 2 (1987), 130–143.
- [10] Ethan Buchman. 2016. *Tendermint: Byzantine fault tolerance in the age of blockchains*. Ph. D. Dissertation. University of Guelph.
- [11] Christian Cachin, Rachid Guerraoui, and Luis Rodrigues. 2011. *Introduction to reliable and secure distributed programming*. Springer Science & Business Media.
- [12] Christian Cachin and Stefano Tessaro. 2005. Asynchronous verifiable information dispersal. In *24th IEEE Symposium on Reliable Distributed Systems (SRDS'05)*. IEEE, 191–201.
- [13] Miguel Castro, Barbara Liskov, et al. 1999. Practical byzantine fault tolerance. In *OSDI*, Vol. 99. 173–186.
- [14] Shir Cohen, Rati Gelashvili, Lefteris Kokoris-Kogias, Zekun Li, Dahlia Malkhi, Alberto Sonnino, and Alexander Spiegelman. 2022. Be aware of your leaders. In *International Conference on Financial Cryptography and Data Security*. Springer, 279–295.
- [15] Shir Cohen, Guy Goren, Lefteris Kokoris-Kogias, Alberto Sonnino, and Alexander Spiegelman. 2023. Proof of availability and retrieval in a modular blockchain architecture. In *International Conference on Financial Cryptography and Data Security*. Springer, 36–53.
- [16] George Danezis, Lefteris Kokoris-Kogias, Alberto Sonnino, and Alexander Spiegelman. 2022. Narwhal and tusk: a dag-based mempool and efficient bft consensus. In *Proceedings of the Seventeenth European Conference on Computer Systems*. 34–50.
- [17] Sourav Das, Zhuolun Xiang, and Ling Ren. 2021. Asynchronous data dissemination and its applications. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*. 2705–2721.
- [18] Henry de Valence. 2024. Ed25519 for consensus-critical contexts. <https://crates.io/crates/ed25519-consensus>.
- [19] Diem Association. 2025. Diem Blockchain. <https://github.com/diem/diem>. Accessed: 2025.
- [20] Die.Net. 2024. writev(3) - Linux man page. <https://linux.die.net/man/3/writev>.
- [21] Sisi Duan, Haibin Zhang, Xiao Sui, Baohan Huang, Changchun Mu, Gang Di, and Xiaoyun Wang. 2024. Dashing and Star: Byzantine fault tolerance with weak certificates. In *Proceedings of the Nineteenth European Conference on Computer Systems*. 250–264.
- [22] Cynthia Dwork, Nancy Lynch, and Larry Stockmeyer. 1988. Consensus in the Presence of Partial Synchrony. *J. ACM* 35, 2 (1988), 288–323. doi:10.1145/42282.42283
- [23] Neil Girdharan. 2025. Autobahn Artifact. <https://github.com/neilgiri/autobahn-artifact>. Accessed: 2025.
- [24] Neil Girdharan, Florian Suri-Payer, Ittai Abraham, Lorenzo Alvisi, and Natacha Crooks. 2024. Autobahn: Seamless high speed BFT. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles*. 1–23.
- [25] Oded Green. 2021. HashGraph—Scalable hash tables using a sparse graph data structure. *ACM Transactions on Parallel Computing (TOPC)* 8, 2 (2021), 1–17.
- [26] Idit Keidar, Oded Naor, Ouri Poupko, and Ehud Shapiro. 2023. Cordial Miners: Fast and Efficient Consensus for Every Eventuality. doi:10.4230/LIPICS.DISC.2023.26
- [27] Mysten Labs. 2024. Mysticeti: Low-latency dag consensus with fast commit path. <https://github.com/asonnino/mysticeti>.
- [28] Meta. 2024. Sapling (Minibytes). <https://github.com/facebook/sapling/tree/main/eden/scm/lib/minibytes>.
- [29] MystenLabs. 2025. Sui Blockchain. <https://github.com/mystenlabs/sui>. Accessed: 2025.
- [30] Nibesh Shrestha. 2025. Sailfish Codebase. <https://github.com/nibeshshrestha/sailfish>. Accessed: 2025.
- [31] Qianyu Yu. 2025. Sailfish++ Codebase. <https://github.com/qyu100/SFSailfish/tree/OptSFSailfish>. Accessed: 2025.
- [32] Michael O Rabin. 1989. Efficient dispersal of information for security, load balancing, and fault tolerance. *Journal of the ACM (JACM)* 36, 2 (1989), 335–348.
- [33] RustCrypto. 2024. RustCrypto: Hashes. <https://github.com/RustCrypto/hashes>.
- [34] Ehud Shapiro. 2024. Grassroots Systems: Concept, Examples, Implementation and Applications. arXiv:2301.04391 [cs.NI] <https://arxiv.org/abs/2301.04391>
- [35] Nibesh Shrestha and Aniket Kate. 2026. Towards Improving Throughput and Scalability of DAG-based BFT SMR. In *Proceedings of the 21st European Conference on Computer Systems*.
- [36] Nibesh Shrestha, Rohan Shrothrium, Aniket Kate, and Kartik Nayak. 2024. Sailfish: Towards Improving the Latency of DAG-based BFT. *Cryptology ePrint Archive* (2024).
- [37] Nibesh Shrestha, Qianyu Yu, Aniket Kate, Giuliano Losa, Kartik Nayak, and Xuechao Wang. 2025. Optimistic, Signature-Free Reliable Broadcast and Its Applications. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*.
- [38] Alberto Sonnino. 2025. Bullshark Implementation. <https://github.com/asonnino/narwhal/tree/bullshark>. Accessed: 2025.
- [39] Alberto Sonnino. 2025. HotStuff Implementation. <https://github.com/asonnino/hotstuff/tree/3-chain>. Accessed: 2025.
- [40] Alberto Sonnino. 2025. Jolteon Implementation. <https://github.com/asonnino/hotstuff>. Accessed: 2025.
- [41] Alberto Sonnino. 2025. Narwhal and Tusk Implementation. <https://github.com/asonnino/narwhal>. Accessed: 2025.
- [42] Alexander Spiegelman, Balaji Arun, Rati Gelashvili, and Zekun Li. 2025. Shoal: Improving DAG-BFT Latency and Robustness. In *Financial Cryptography and Data Security (FC 2024), Revised Selected Papers, Part I (Lecture Notes in Computer Science, Vol. 14744)*. Springer, Cham, 92–109. doi:10.1007/978-3-031-78676-1_6 FC 2024, Willemstad, Curaçao, March 4–8, 2024.
- [43] Alexander Spiegelman, Neil Girdharan, Alberto Sonnino, and Lefteris Kokoris-Kogias. 2022. Bullshark: Dag bft protocols made practical. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*. 2705–2718.
- [44] The IOTA team. 2025. <https://docs.iota.org/about-iota/iota-architecture/consensus>.
- [45] The Sui Team. 2025. Validator Deployment amd Configuration. <https://docs.sui.io/guides/operator/validator/validator-config>.
- [46] The Tokio Team. 2024. Tokio. <https://tokio.rs>.
- [47] Pasindu Tennage. 2025. Mahi-Mahi Consensus Implementation. <https://github.com/PasinduTennage/mahi-mahi-consensus>. Accessed: 2025.
- [48] Giorgos Tsimos, Anastasios Kichidis, Alberto Sonnino, and Lefteris Kokoris-Kogias. 2023. HammerHead: Leader Reputation for Dynamic Scheduling. arXiv:2309.12713 [cs.CR] <https://arxiv.org/abs/2309.12713>
- [49] Dimitris Vyzovitis, Yusef Napora, Dirk McCormick, David Dias, and Yiannis Psaras. 2020. *GossipSub: Attack-Resilient Message Propagation in the Filecoin and ETH2.0 Networks*. Technical Report PL-TechRep-2020-002. Protocol Labs. <https://arxiv.org/abs/2007.02754>.
- [50] Daniel Xiang. 2025. Ditto Implementation. <https://github.com/danielxiangz/ditto>. Accessed: 2025.
- [51] Lei Yang, Vivek Bagaria, Gerui Wang, Mohammad Alizadeh, David Tse, Giulia Fanti, and Pramod Viswanath. 2019. Prism: Scaling bitcoin by 10,000 x. *arXiv preprint arXiv:1909.11261* (2019).
- [52] Jianting Zhang and Aniket Kate. 2025. No fish is too big for flash boys! frontrunning on DAG-based blockchains. In *2025 IEEE Annual Computer Security Applications Conference (ACSAC)*. IEEE, 1065–1080.

A Open Science

All artifacts necessary for evaluating the contributions of this paper are publicly available at:

<https://anonymous.4open.science/r/beluga-0F4F>

Appendix A.1 additionally provides a tutorial to reproduce experiments.

A.1 Reproducing Experiments

We provide the orchestration scripts² used to benchmark the Beluga codebase on AWS and produce the benchmarks of Section 6.

Deploying a testbed The file ‘`~/aws/credentials`’ should have the following content:

```
[default]
aws_access_key_id = YOUR_ACCESS_KEY_ID
aws_secret_access_key = YOUR_SECRET_ACCESS_KEY
```

configured with account-specific AWS *access key id* and *secret access key*. It is advise to not specify any AWS region as the orchestration scripts need to handle multiple regions programmatically.

A file ‘`settings.yaml`’ contains all the configuration parameters for the testbed deployment. We run the experiments of Section 6 with the following settings:

```
---
testbed_id: "${USER}-testbed"
cloud_provider: aws
token_file: "/Users/${USER}/.aws/credentials"
ssh_private_key_file: "/Users/${USER}/.ssh/aws"
regions:
  - us-east-1
  - us-west-2
  - ca-central-1
  - eu-central-1
  - eu-west-1
  - eu-west-2
  - eu-west-3
  - eu-north-1
  - ap-south-1
  - ap-southeast-1
  - ap-southeast-2
  - ap-northeast-1
  - ap-northeast-2
specs: m5d.8.xlarge
repository:
  url: https://github.com/AUTHOR/REPO.git
  commit: main
node_parameters_path:
  "crates/orchestrator/assets/node-parameters.yml"
client_parameters_path:
  "crates/orchestrator/assets/client-parameters.yml"
benchmark_duration: 1000
```

where the file ‘`/Users/${USER}/.ssh/aws`’ holds the ssh private key used to access the AWS instances, and ‘`AUTHOR`’ and ‘`REPO`’ are respectively the GitHub username and repository name of the codebase to benchmark.

The orchestrator binary provides various functionalities for creating, starting, stopping, and destroying instances. For instance, the following command to boots 2 instances per region (if the settings file specifies 13 regions, as shown in the example above, a total of 26 instances will be created):

```
cargo run --bin orchestrator -- testbed deploy --instances 2
```

The following command displays the current status of the testbed instances

```
cargo run --bin orchestrator testbed status
```

Instances listed with a green number are available and ready for use and instances listed with a red number are stopped. It is necessary to boot at least one instance per load generator, one instance per validator, and one additional instance for monitoring purposes (see below). The following commands respectively start and stop instances:

```
cargo run --bin orchestrator -- testbed start
cargo run --bin orchestrator -- testbed stop
```

² <https://anonymous.4open.science/r/beluga-0F4F>

It is advised to always stop machines when unused to avoid incurring in unnecessary costs.

Running Benchmarks Running benchmarks involves installing the specified version of the codebase on all remote machines and running one validator and one load generator per instance. For example, the following command benchmarks a committee of 50 validators under a constant load of 1,000 tx/s:

```
cargo run --bin orchestrator -- benchmark \
  --committee 50 fixed-load --loads 1000
```

The nodes and clients configuration files (respectively specified in ‘`crates/orchestrator/assets/node-parameters.yml`’ and ‘`crates/orchestrator/assets/client-parameters.yml`’) are used to respectively parametrize the nodes and clients. We run our benchmarks with the nodes configuration file as follows:

```
leader_timeout:
  secs: 1
  nanos: 0
```

and the client configuration file as follows:

```
initial_delay:
  secs: 400
  nanos: 0
```

Monitoring The orchestrator provides facilities to monitor metrics. It deploys a Prometheus instance and a Grafana instance on a dedicated remote machine. Grafana is then available on the address printed on stdout when running benchmarks with the default username and password both set to admin. An example Grafana dashboard can be found in the file ‘`grafana-dashboard.json`’³.

B Ethical Considerations

This work presents a pull-induction attack and shows how it can be launched to degrade the performance of many modern BFT consensus protocols. We conduct our attack evaluation in a self-deployed environment in a controlled way, which does not affect any production systems. While the present attacks might be exploited in real-world systems that have adopted the corresponding protocols, the primary contribution is a mitigation. We have disclosed this vulnerability to the relevant Sui team and helped them integrate our mitigation into their system.

C Performance Analysis

We prove the round latency of Beluga both in the happy case and under adversarial setting.

C.1 Performance Analysis in the Happy Case

In this section, we will show that Beluga can achieve optimal round latency (i.e., Δ) under happy cases and the round latency of nearly 2Δ under adverse cases after GST.

In happy cases, all responsive validators (at least $2f+1$) are honest and share their blocks in time. According to Lemma 1, all honest validators can enter the same round (w.l.o.g. at round r) within 3Δ after GST and can create their round r blocks by time $GST+3\Delta$. As a result, every honest validator must be able to receive a quorum $\mathcal{B}^r$ containing at least $2f+1$ round r blocks by time $GST+4\Delta$. Since all responsive validators are honest in happy cases, every honest validator will receive $\mathcal{B}^r$ at time, w.l.o.g., $t_{sync} < GST+4\Delta$ and move

³ <https://anonymous.4open.science/r/beluga-0F4F/crates/orchestrator/assets/grafana-dashboard.json>

to the next round $r+1$ at t_{sync} . After that, all honest validators can receive at least $2f+1$ round $r+1$ blocks by time $t_{sync}+\Delta$ and move to round $r+2$ at time $t_{sync}+\Delta$. By induction, we can see that the round latency under happy cases is Δ .

C.2 Performance Analysis under Adverse cases

In this section, we give a rigorous proof to show that Beluga can achieve a round latency of nearly 2Δ under adverse cases. The proof relies on the following assumption. (Note that the correctness of Beluga (cf. Section 5) does not rely on these assumptions.)

ASSUMPTION 1 (LATENCY TRIANGLE). *After GST, the direct network latency between any pair of honest validators is always faster than going through an intermediate validator.*

In adverse cases, there are at most f malicious validators that aim to increase the round latency by inducing honest validators to trigger the pull protocol. In the following, we denote the set of honest validators as $\mathcal{V}_h$ and the set of malicious validators as $\mathcal{A}$.

LEMMA 3. *After GST, each honest validator will not get blamed and have its reputation decreased by honest validators.*

PROOF. Recall from Section 4.2 that a validator v_i has its reputation decreased by honest validators only if $f+1$ validators report having invoked the pull protocol to synchronize v_i 's blocks. An honest validator v_j invokes the pull protocol upon receiving a block B_k^r from a validator v_k that references v_i 's round $r-1$ block B_i^{r-1} , while v_j has not yet received B_i^{r-1} directly. By Assumption 1, however, after GST, if v_i is honest and sends B_i^{r-1} to v_j , then v_j receives B_i^{r-1} directly from v_i before receiving it through any intermediate validator v_k that references it in B_k^r . Hence, v_j does not invoke the pull protocol for B_i^{r-1} and does not report v_i . Since there are at most f malicious validators, an honest v_i cannot be reported by $f+1$ validators and therefore cannot be blamed by honest validators. $\square$

LEMMA 4. *After GST, if all honest validators enter round r at time t_r and have their reputation higher than that of any malicious validator, then for any future round $r' \geq r$, the latency of round r' is Δ .*

PROOF. Since all honest validators have higher reputation than any malicious validator, by the reputation-based push protocol, honest validators reference only round $r-1$ blocks from honest validators when creating their round r blocks. After GST, these round r blocks are accepted without invoking the pull protocol. Consequently, round r has latency Δ , and all honest validators enter round $r+1$ at time $t_r+\Delta$. By induction, the latency of any future round $r' \geq r$ is Δ . $\square$

LEMMA 5. *After GST, if all honest validators enter round r at time t_r , then either the expected latency of any future round $r' > r$ is within 2Δ or at least one malicious validator is blamed by honest validators, in which case the latency of round r' is at most 3Δ .*

PROOF. Recall that if an honest validator v_i enters round r at time t_r , then v_i must have received at least $2f+1$ acceptable round $r-1$ blocks and can create its round r block at t_r . Since all honest validators enter round r at time t_r , every honest validator will receive at least $2f+1$ round r blocks created by honest validators by time $t_r+\Delta$. There are three cases.

Case 1: If all honest validators have their reputation higher than that of any malicious validator, then according to Lemma 4, the latency of future round $r' > r$ is Δ .

Case 2: If it is not the case 1, and for each round $r-1$ block created by malicious validators $\mathcal{A}$, it is referenced by at least $f+1$ round r blocks created by honest validators, i.e., $\mathcal{A}$ share their round $r-1$ blocks with at least $f+1$ honest validator. In this case, thanks to the ImPoA-based pull mechanism, every honest validator can accept all round r blocks created by honest validators without synchronizing any missing blocks on the push path and create its round $r+1$ block at time $t_r+\Delta$. The latency of round $r+1$ is Δ . By induction, we can see that for any future round $r' \geq r$, the latency of round r' is Δ for case 2.

Case 3: If it is not the case 1, and at least one round $r-1$ block B_m^{r-1} created by a malicious validator v_m is referenced by fewer than $f+1$ round r blocks created by honest validators, i.e., v_m delays or did not share B_m^{r-1} with more than f honest validators. We denote those honest validators referencing B_m^{r-1} by t_r as $\mathcal{V}_h^{Ref}$, and those who do not as $\mathcal{V}_h^{NoR}$. There are two scenarios, and we show that for any future round $r' \geq r+1$, the expected latency of r' is at most 2Δ before v_m is blamed.

First, v_m will never share B_m^{r-1} with $\mathcal{V}_h^{NoR}$, then $|\mathcal{V}_h^{NoR}| \geq f+1$ honest validators will report v_m , and v_m will be blamed by honest validators.

Second, v_m delays sharing B_m^{r-1} with some honest validators $\mathcal{V}_{h1}^{NoR}$ but not the others $\mathcal{V}_{h2}^{NoR}$ to escape being blamed, where $|\mathcal{V}_{h2}^{NoR} = \mathcal{V}_h^{NoR} \setminus \mathcal{V}_{h1}^{NoR}| \leq f$. In this scenario, note that $\mathcal{V}_{h1}^{NoR}$ must have received B_m^{r-1} by time $t_r+\Delta$, since otherwise, $\mathcal{V}_{h1}^{NoR}$ learn B_m^{r-1} is missing from $\mathcal{V}_h^{Ref}$'s round r blocks and will report v_m . Thus, both $\mathcal{V}_h^{Ref}$ and $\mathcal{V}_{h1}^{NoR}$ can create their round $r+1$ blocks by time $t_r+\Delta$. The delayed honest validators $\mathcal{V}_{h2}^{NoR}$, instead, must pull B_m^{r-1} at time $t_r+\Delta$ (when they receive round r blocks from $\mathcal{V}_h^{Ref} \cup \mathcal{V}_{h1}^{NoR}$) and pull any missing round r blocks at time $t_r+2\Delta$ (when they receive round $r+1$ blocks from $\mathcal{V}_h^{Ref} \cup \mathcal{V}_{h1}^{NoR}$). As a result, even for the delayed honest validators $\mathcal{V}_{h2}^{NoR}$, they can create their round $r+1$ blocks by time $t_r+3\Delta$ and create their round $r+2$ blocks $t_r+4\Delta$. Since the non-delayed $\mathcal{V}_h^{Ref} \cup \mathcal{V}_{h1}^{NoR}$ can create their round $r+2$ blocks by time $t_r+4\Delta$, such a delaying process can be repeated every two rounds. As a result, the maximum average latency of each future round $r' \geq r+1$ is delayed by at most 2Δ , if v_m wishes to escape being blamed.

In addition, when v_m is blamed due to it delaying a round r'' , according to Lemma 2, the latency of round r'' is at most 3Δ . The proof is done. $\square$

LEMMA 6. *After GST, once all honest validators enter round r , for future round $r' > r$ that malicious validators $\mathcal{A}$ delay, its expected latency is within 2Δ , or at least one malicious validator is blamed by honest validators.*

PROOF. By Lemma 1, all honest validators enter a common round within 3Δ after GST; w.l.o.g. call this round r and let $t_r := GST+3\Delta$. Moreover, at least one honest validator v_{fst} must have created its

round r block B_{fst}^r by time GST , so every honest validator receives B_{fst}^r by time $GST+\Delta$.

Consider two sets of honest validators: slow honest validators $\mathcal{V}_h^{slw}$ and fast honest validators $\mathcal{V}_h^{fst} = \mathcal{V}_h \setminus \mathcal{V}_h^{slw}$. Validators in $\mathcal{V}_h^{slw}$ need to invoke the pull protocol for missing round $r-1$ blocks to enter round r by time t_r , while validators in $\mathcal{V}_h^{fst}$ do not. Apparently, we have $|\mathcal{V}_h^{fst}| \geq f+1$, since otherwise, there are more than $f+1$ honest validators in $\mathcal{V}_h^{slw}$ reporting $\mathcal{A}$, and $\mathcal{A}$ would be blamed. Moreover, note that all validators in $\mathcal{V}_h^{fst}$ must be able to create their round r blocks by time $GST+\Delta$ (i.e., right after receiving B_{fst}^r), since otherwise, they need to invoke the pull protocol to fetch missing round $r-1$ blocks referenced by B_{fst}^r , contradicting to the definition of $\mathcal{V}_h^{fst}$. As a result, all validators in $\mathcal{V}_h^{slw}$ must receive round r blocks from all validators in $\mathcal{V}_h^{fst}$ by time $GST+2\Delta$. With our pull protocol ensuring all live blocks can be acceptable within 2Δ , all validators in $\mathcal{V}_h^{slw}$ can accept all round r blocks from $\mathcal{V}_h^{fst}$ by $GST+4\Delta = t_r+\Delta$. Consequently, validators in $\mathcal{V}_h^{slw}$ can create their round $r+1$ blocks at time $t_r+\Delta$. In addition, because validators in $\mathcal{V}_h^{fst}$ can receive $\mathcal{V}_h^{slw}$'s round r blocks by time $t_r+\Delta$, they must be able to create their round $r+1$ blocks by time $t_r+\Delta$. The latency of round $r+1$ is Δ . For the next round $r+2$, there are two cases.

Case 1: If all honest validators enter round $r+1$ at time $t_r+\Delta$, according to Lemma 5, either the expected latency of round $r+2$ is within 2Δ or at least one malicious validator is blamed by honest validators.

Case 2: Otherwise, honest validators in $\mathcal{V}_h^{fst}$ enter round $r+1$ before they receive $\mathcal{V}_h^{slw}$'s round r blocks at $t_r+\Delta$. There are two scenarios, and we show that the expected latency of round $r+2$ is within 2Δ before at least one malicious validator is blamed.

First, every round r block created by malicious validators $\mathcal{A}$ is referenced by at least $f+1$ round $r+1$ blocks created by honest validators in $\mathcal{V}_h^{fst}$. In this scenario, thanks to the ImPoA-based pull mechanism, every honest validator in $\mathcal{V}_h^{slw}$ can accept these round $r+1$ blocks without synchronizing any missing blocks on the push path. As a result, $\mathcal{V}_h^{slw}$ can create their round $r+2$ blocks at time $t_r+2\Delta$. Since, $\mathcal{V}_h^{slw}$ create their round $r+1$ blocks at time $t_r+\Delta$ (as mentioned above), the latency of round $r+2$ is within Δ .

Second, at least one round r block created by malicious validators $\mathcal{A}$ is referenced by fewer than $f+1$ round $r+1$ blocks created by honest validators in $\mathcal{V}_h^{fst}$. This means that at least one malicious validator v_m delays sharing its round r block B_m^r with some validators in $\mathcal{V}_h^{fst} \subset \mathcal{V}_h^{fst}$, because otherwise, all honest validators in $\mathcal{V}_h^{fst}$ can receive enough round r blocks and enter round $r+1$ at the same time. However, all honest validators in $\mathcal{V}_h^{fst}$ must be able to create round $r+2$ blocks by time $t_r+2\Delta$, since the delayed validators in $\mathcal{V}_h^{slw}$ create their round $r+1$ blocks at time $t_r+\Delta$ (as mentioned above). As a result, once receiving $\mathcal{V}_h^{fst}$'s round $r+2$ blocks at time $t_r+3\Delta$, $\mathcal{V}_h^{slw}$ can invoke the pull protocol to fetch any missing round $r+1$ blocks within 2Δ , and create their round

$r+3$ blocks by time $t_r+5\Delta$. Recall that $\mathcal{V}_h^{slw}$ create their round $r+1$ blocks at time $t_r+\Delta$. For $\mathcal{V}_h^{slw}$, there are two rounds $r+1$ and $r+2$ between time $t_r+\Delta$ and $t_r+5\Delta$. As a result, the average latency of these two rounds is $(t_r+5\Delta - t_r+\Delta)/2 = 2\Delta$.

By induction, we can see that for any future round $r'>r$, the latency of round r' is within 2Δ before any malicious validator v_m is blamed. The proof is done. $\square$

LEMMA 7. *After GST, malicious validators $\mathcal{A}$ in Beluga can only delay the progress of the protocol in a bounded number of rounds with an expected latency higher than 2Δ every R_L rounds.*

PROOF. Recall that, when blamed, a validator has its reputation decreased by R_L by every honest validator, where $R_L \geq 1$ is a predetermined parameter. Without loss of generality, assume that right after GST each malicious validator has reputation R_m , and the lowest reputation of any honest validator is R_h .

By Lemma 6, $\mathcal{A}$ can delay the protocol for one round with latency at most 4Δ without being blamed. Denote this period by $\mathcal{D}_1$, and let $|\mathcal{D}_1|$ denote the number of rounds it delays with expected latency exceeding 2Δ . Thus, $\mathcal{D}_1$ adds at most 3Δ to the total latency before $\mathcal{A}$ is blamed.

After period $\mathcal{D}_1$, all honest validators can enter the same round at the same time. According to Lemma 5, $\mathcal{A}$ might still be able to delay some future rounds with the expected latency of more than 2Δ , but each of such round will lead to at least one malicious validator being blamed and getting lose of R_L points. Based on the reputation difference, we can derive that f malicious validators can delay the protocol for $(R_m - R_h) \cdot f / R_L$ rounds before their reputations are lower than that of any honest validator. We denote this period as $\mathcal{D}_2$, and similarly, we have $|\mathcal{D}_2| = (R_m - R_h) \cdot f / R_L$. According to Lemma 5, each of these rounds will increase the round latency by at most 2Δ . As a result, $\mathcal{D}_2$ will increase the latency of the protocol by at most $(R_m - R_h) \cdot f / R_L \cdot 2\Delta$.

After period $\mathcal{D}_2$, $\mathcal{A}$ have the reputation equal to the lowest reputation of an honest validator. Since according to Lemma 3, the honest validators will not get their reputations decreased after GST, $\mathcal{A}$ need to perform correctly to get their reputation increased by honest validators. In particular, to delay f rounds with the expected latency of more than 2Δ for each after $\mathcal{D}_2$, these f malicious validators $\mathcal{A}$ perform carefully without getting blamed for at least R_L rounds, during which the expected latency of each round will be 2Δ . We denote this period as $\mathcal{D}_3$. During $\mathcal{D}_3$, $\mathcal{A}$ delay f rounds with the expected latency of higher than 2Δ every R_L rounds. Thus, we have $|\mathcal{D}_3| = f$. According to Lemma 5, each round being delayed will increase the round latency by at most 2Δ . As a result, $\mathcal{D}_3$ will increase the latency of the protocol by at most $f \cdot 2\Delta$ every R_L rounds.

By considering the above three periods, we can conclude that $\mathcal{A}$ can delay the protocol with the expected round latency higher than 2Δ for at most $|\mathcal{D}_1|+|\mathcal{D}_2|+|\mathcal{D}_3| = 1+(R_m - R_h) \cdot f / R_L + f$ rounds, and the extra latency introduced by $\mathcal{A}$ is at most $(3\Delta + (R_m - R_h) \cdot f / R_L \cdot 2\Delta + f \cdot 2\Delta)$ every R_L rounds. As f, R_m, R_h , and R_L are all constants, the proof is done. $\square$

Finally, we have a proof for Theorem 5.

THEOREM 5. *After GST, Beluga can achieve a round latency of nearly 2Δ under adverse cases.*

PROOF FOR THEOREM 5. According to Lemma 7, for every R_L rounds, malicious validators $\mathcal{A}$ can only increase the round latency by more than 2Δ for a bounded number of rounds (corresponding to the $\mathcal{D}_1$, $\mathcal{D}_2$, and $\mathcal{D}_3$ periods). For the remaining rounds, the expected round latency is 2Δ . Hence, the total latency over R_L rounds can be upper-bounded by $2\Delta + \Delta_{\text{extra}}$, where Δ_{extra} denotes the additional delay introduced by the adversarial periods.

From Lemma 7, this extra delay satisfies $\Delta_{\text{extra}} \leq 3\Delta + \frac{(R_m - R_h)f}{R_L} \cdot 2\Delta + f \cdot 2\Delta$. Therefore, the average round latency over these R_L rounds is at most $\frac{R_L \cdot 2\Delta + \Delta_{\text{extra}}}{R_L} = 2\Delta + \frac{\Delta_{\text{extra}}}{R_L}$. Substituting the bound on Δ_{extra} , we obtain $2\Delta + \frac{3\Delta}{R_L} + \frac{(R_m - R_h)f}{R_L^2} \cdot 2\Delta + \frac{f}{R_L} \cdot 2\Delta$.

Observe that the delays caused by the $\mathcal{D}_1$ (corresponding to $\frac{3\Delta}{R_L}$ term) and $\mathcal{D}_2$ (corresponding to $\frac{(R_m - R_h)f}{R_L^2} \cdot 2\Delta$ term) periods occur only once, and thus become negligible over a sufficiently long execution. Hence, asymptotically, the average round latency is $2\Delta(1 + \frac{f}{R_L}) = 2\kappa\Delta$ with $\kappa = 1 + \frac{f}{R_L}$. By setting a sufficiently large R_L , the average round latency can be arbitrarily close to 2Δ . $\square$

D Security Analysis of Mysticeti-Beluga

Section 4.4 discusses how to build a BFT consensus on Beluga to achieve a more efficient and robust block synchronization, and in Section 6, we instantiate Mysticeti consensus over Beluga to obtain a variant consensus Mysticeti-Beluga, demonstrating the performance improvement in practice. This section gives the security proof of Mysticeti-Beluga.

Like most BFT consensus protocols, we consider two security properties for Mysticeti-Beluga: *liveness* and *safety*. Similar to Mysticeti, the security properties of Mysticeti-Beluga only rely on strong links.⁴ For simplicity, all references and links used in this section are considered strong links if not stated otherwise. To help readers better follow the proof and make the section self-contained, we first review the consensus logic of Mysticeti and show how to integrate Beluga into Mysticeti (Appendix D.1), and then prove the liveness (Appendix D.2) and safety (Appendix D.3) of Mysticeti-Beluga.

D.1 Build Mysticeti on Beluga

D.1.1 Consensus Logic in Mysticeti. Mysticeti orders transactions on top of the DAG constructed by its block synchroniser module using a two-step scheme. Below, we give the necessary definitions and notations required by the security proof, but recommend readers refer to Mysticeti's paper for more details about its consensus logic.

(1) Decide status of leaders. First, Mysticeti designates leader validators using a round-robin approach⁵ and employs two decision rules to decide the statuses of leader blocks that are proposed by the designated leader validators. To achieve this, Mysticeti defines two DAG patterns on a leader block B_L : (i) *skip pattern*, if B_L is not referenced by $2f+1$ blocks of the successive round; (ii) *certificate*

pattern,⁶ if B_L is referenced by at least $2f+1$ blocks of the next round. Any subsequent block that includes a certificate pattern on B_L via strong links in its causal history is called a *certificate* for B_L and is considered to certify B_L . For instance, in Figure 3(a), the round $r-1$ block B_2^{r-1} is referenced by $\{B_1^r, B_2^r, B_4^r\}$ from round r , and thus forms a certificate pattern. Since B_2^{r+1} from round $r+1$ references $\{B_1^r, B_2^r, B_4^r\}$, B_2^{r+1} is a certificate for B_2^{r-1} .

Validators then employ the following decision rules to decide leader blocks as *to-commit* status or *to-skip* status:

- **Direct decision rule:** A round r leader block B_L^r is directly decided if it (i) has at least $2f+1$ certificate patterns at round $r+2$, in which B_L^r is decided as *to-commit*, or (ii) is identified as a *skip pattern*, in which B_L^r is decided as *to-skip*. Otherwise, it remains undecided.
- **Indirect decision rule:** For any undecided round r' leader block $B_L^{r'}$, a replica searches for the first subsequent leader block $B_L^{r''}$ (where $r'' > r' + 2$) that is either decided as *to-commit* or is still undecided. In the former case, if $B_L^{r''}$ causally references a certificate for $B_L^{r'}$, then $B_L^{r'}$ is decided as *to-commit*; otherwise, it is decided as *to-skip*. In the later case, $B_L^{r'}$ remains undecided for the moment (Mysticeti guarantees that it will eventually be decided).

If a leader block is decided as *to-commit*, we call that the validators commit the leader block. Similarly, if a leader block is decided as *to-skip*, we call that the validators skip the leader block.

(2) Order transactions. Second, after all leader blocks up to round r are decided, validators order all leader blocks up to round r that have been decided as *to-commit* and their causal history blocks using a deterministic linearization algorithm, e.g., a breadth-first search traversal [52]. Recall that in the Mysticeti's original block synchronizer module (i.e., DAG-based uncertified synchronizer protocol), validators reference a leader block only when they receive *all transactions* of the leader block's causal history. As a result, once blocks are ordered, transactions in the blocks are ordered and can be finalized as well. Here, the transaction finalization is defined based on the specific protocols/applications. For instance, in the state machine replication (SMR), transaction finalization indicates that the transaction is executed. Nevertheless, they all require that validators have full transaction data.

Leader timeout. Since Mysticeti operates in the partial synchrony network model, it relies on timeouts to ensure liveness after GST. Mysticeti's consensus logic introduces a leader timeout T_{ld} to round advancement. In particular, validators *is ready* to advance to round r (i) upon receiving the round $r-1$ leader block and receiving $2f+1$ round $r-1$ blocks referencing the round $r-2$ leader block, or (ii) upon the expiration of T_{ld} .

D.1.2 Integrate Beluga into Mysticeti. By replacing Mysticeti's original synchronizer module with Beluga, we can derive a variant consensus protocol Mysticeti-Beluga. Below, we demonstrate how Beluga interfaces with Mysticeti's consensus logic (Figure 4 presents the overview of the interfaces). It is worth noting that Mysticeti-Beluga does not modify Mysticeti's consensus logic.

⁴Mysticeti only defines strong links (or parents) but not weak links.

⁵We consider single-leader when building Mysticeti-Beluga, where each round contains only one leader.

⁶The same pattern as we defined in Sec. 4.4 of this paper.

Round advancement. In Mysticeti-Beluga, a validator v_i advances to round r and calls $block_propose_i(B, r)$ to create and disseminate a round r block B to the system. Note that both Beluga and Mysticeti's consensus logic define rules for round advancement. Mysticeti-Beluga incorporates all these rules to preserve the relevant properties. Specifically, v_i calls $block_propose_i(B, r)$ if one of the following round advancement rules becomes satisfied: (i) v_i receives $2f+1$ blocks from round $r-1$ whose creators have reputations above the threshold R_t and receives the round $r-1$ leader block B_L^{r-1} and receives $2f+1$ round $r-1$ blocks referencing B_L^{r-2} ; or (ii) a timeout $T_{live} = \max\{T_{ld}, T_{rd}\}$ expires, where T_{ld} is the leader timeout introduced by Mysticeti's consensus logic, and T_{rd} is the pre-round timeout used by Beluga for its round advancements in the synchronizer module. Note that T_{ld} is set to 4Δ [6, Lemma 9] in Mysticeti, same as T_{rd} set in Beluga (Sec. 4.2), and both T_{ld} and T_{rd} are per-round timeouts (i.e., v_i reschedules them once advancing to a new round). As a result, we use T_{live} solely in our liveness proof for illustration purposes.

Parent selection. When creating a new block, v_i refers to its admission control rules to prioritize selecting high-reputation blocks that have been output via $block_accept$ and are considered *consensus-specified*. A block in round $r-1$ is considered consensus-specified if it references the round $r-2$ leader block. Note that including consensus-specified blocks as parents is required by Mysticeti's consensus logic (for liveness) and is not inherently required by Beluga. Nevertheless, Beluga can help validators filter out consensus-specified blocks that are created by suspected Byzantine validators and prevent them from delaying the progress of pushing blocks.

Block ordering and transaction finalization. When ordering blocks, v_i executes Mysticeti's decision rules on leader blocks that have been output via $block_accept$. In particular, similar to the vanilla Mysticeti, Mysticeti-Beluga employs a two-step scheme to order blocks. First, validators decide the status of every leader block using the two underlying decision rules. Then, validators order all leader blocks that have been decided as *to-commit* status and their causal history blocks. Transactions in a block B in Mysticeti-Beluga are finalized if *all* blocks that are ordered before B have been output via $block_store$ and their transactions have been finalized.

D.2 Liveness of Mysticeti-Beluga

We first show that Mysticeti-Beluga ensures liveness under partial synchrony. The liveness states that transactions will eventually be ordered and finalized by validators.

LEMMA 8. *In Mysticeti-Beluga, after GST, an honest validator's leader block will be referenced in the next round by every honest validator.*

PROOF. After GST, if an honest validator enters a round r , then the honest leader validator (and every other honest validator) will be able to enter the same round r within 3Δ (Lemma 1). Then the honest leader validator will directly create and disseminate the round r leader block B_L^r , which will take another Δ to be received by every honest validator. Since the timeout T_{live} is set to 4Δ , B_L^r will arrive before the first honest validator times out. Since validators are asked to include leader blocks as parents (see Appendix D.1.2 and Sec. 4.4), every honest validator will vote for the leader block. $\square$

LEMMA 9. *In Mysticeti-Beluga, after GST, all honest validators will create a certificate for the leader block proposed by an honest validator.*

PROOF. Assume there is an honest leader block B_L^r in round r , by Lemma 8, all honest validators will vote for B_L^r after GST. This means that B_L^r is a certified block, and all honest validators will have their round $r+1$ blocks referencing B_L^r as parents. By Lemma 1, every honest validator can receive $2f+1$ honest blocks from round $r+1$ within 4Δ . Consequently, according to the parent selection employed in Mysticeti-Beluga (Appendix D.1.2), where validators wait for 4Δ before giving up the certificate creation in round $r+2$, every honest validator can create a round $r+2$ block that references these $2f+1$ round $r+1$ blocks from honest validators. In other words, every honest validator will create a certificate for B_L^r . $\square$

LEMMA 10. *The round-robin schedule of leader blocks in Mysticeti-Beluga ensures that in any window of $3f+3$ rounds, there are three consecutive rounds with honest leader blocks.*

PROOF. There are $3f+1$ groups of three consecutive rounds. Due to the round-robin schedule, each of the honest validators must be one of the leaders in exactly 3 of these groups. As there are $2f+1$ honest validators, due to the pigeonhole principle, one group must contain $\lceil \frac{3*(2f+1)}{3f+1} \rceil = 3$ honest leader blocks. $\square$

LEMMA 11. *In Mysticeti-Beluga, after GST, any undecided leader block eventually gets decided.*

PROOF. Consider an undecided leader block in round r . After GST, by Lemma 10, there will eventually be three honest leader blocks in three consecutive rounds k , $k+1$, and $k+2$ with $k > r$. By Lemma 9, each of these honest leader blocks will have $2f+1$ certificates and can be decided as *to-commit* via the direct decision rule. We now prove that by induction, all leader blocks in rounds $< k$ get decided. For the base case, any undecided leader blocks in rounds $k-3$, $k-2$, and $k-1$ get decided by the *to-commit* leader blocks in rounds k , $k+1$, and $k+2$, respectively, via the indirect decision rule. For the induction step, any undecided leader block in round $r' < k-3$ also gets decided since k is higher than $r' + 2$ and there are no undecided leader blocks between r' and k (induction hypothesis). $\square$

LEMMA 12. *In Mysticeti-Beluga, if a block B is referenced by $2f+1$ subsequent blocks, then every honest validator will eventually output $block_accept$ for B .*

PROOF. If B is referenced by $2f+1$ subsequent blocks, at least $f+1$ honest validators reference B . These $f+1$ honest blocks $\mathcal{B}_{hst}$ will eventually be received by all honest validators. According to the ImPoA-based pull protocol (Section 4.3), $\mathcal{B}_{hst}$ forms an implicit proof-of-availability certificate for B and is output via $block_accept$ by every honest validator. $\square$

THEOREM 6 (CONSENSUS LIVENESS). *In Mysticeti-Beluga, after GST, transactions will be ordered and finalized.*

PROOF. By Lemma 9, there will be $2f+1$ certificates for each honest leader block after GST, and the honest leader block will be decided as *to-commit*. By Lemma 11, all leader blocks will eventually

get decided. Therefore, validators can order all *to-commit* leader blocks and their causal history blocks. Moreover, since each *to-commit* leader block created is referenced by $2f+1$ subsequent blocks as parents, by Lemma 12, every honest validator will output *block_accept* for the leader block. According to block availability and causal availability ensured by Beluga, the leader block and its causal history blocks will eventually be output via *block_store*. This means that all transactions in *to-commit* leader blocks and their causal history blocks can be retrieved, ordered and finalized. $\square$

D.3 Safety of Mysticeti-Beluga

Beluga only relates to the liveness of protocol and does not interfere with their safety as it does not change their commit rule. Nevertheless, we then show that Mysticeti-Beluga ensures safety. The safety states that all honest validators order transactions in the same order.

LEMMA 13. *In Mysticeti-Beluga, if $2f+1$ round r blocks from distinct validators are certificates of a block B , then every block in any round $r' > r$ must (directly or transitively) reference a certificate for B formed in round r .*

PROOF. Recall that a block is a certificate for B if it references $2f+1$ blocks that themselves reference B . Consider round $r+1$. Every block in this round references $2f+1$ blocks from round r . By quorum intersection, any such set intersects the certificate set of B in at least one honest validator. Since honest validators do not equivocate, every round $r+1$ block must reference a block that is a certificate for B . By induction over rounds, this property propagates to all $r' > r$. $\square$

LEMMA 14. *In Mysticeti-Beluga, if an honest validator directly commits B_L^r , then no honest validator (directly or indirectly) decides to skip B_L^r .*

PROOF. Assume for contradiction that some honest validator commits B_L^r while another skips it.

First, consider B_L^r is directly skipped. A direct skip occurs only if at least $2f+1$ blocks in round $r+1$ do not reference B_L^r . However, if B_L^r is committed, then by the decision rule, at least $2f+1$ round $r+1$ blocks reference it. Any conflicting observation would violate quorum intersection, implying an honest validator equivocated in round $r+1$, a contradiction.

Then, consider B_L^r is indirectly skipped. According to the indirect rule, the skip must arise through a later leader block $B_L^{r'}$ with $r' > r+2$. By the decision rule, such a block must not reference a certificate for B_L^r . However, since B_L^r is directly committed, it has $2f+1$ certificates formed by distinct validators. By Lemma 13, a certificate for B_L^r must be referenced by all future blocks. This contradiction completes the proof. $\square$

LEMMA 15. *In Mysticeti-Beluga, at most one leader block can be certified for any round r .*

PROOF. Suppose two distinct leader blocks B_{L1}^r and B_{L2}^r both obtain $2f+1$ references from round $r+1$. By quorum intersection, at least one honest validator must belong to both quorums, and thus would have referenced both blocks in round r . This contradicts

the protocol rule that a validator references at most one block per proposer per round. $\square$

Lemma 15 implies the following corollary.

COROLLARY 1. *No two honest validators commit distinct leader blocks in the same round.*

LEMMA 16. *In Mysticeti-Beluga, all honest validators decide a consistent status for each round leader block.*

PROOF. Consider two honest validators v_i and v_j , and let n be the highest round in which v_i commits a leader block. We prove by backward induction that for every round $x \leq n$, both validators assign the same status to B_L^x .

Base case ($x=n$). Validator v_i commits B_L^n . By Lemma 14, v_j cannot skip it and must also commit it. By Corollary 1, both commit the same block.

Inductive step. Assume the statement holds for all rounds in $(k, n]$. Consider round k . If either validator directly commits or directly skips B_L^k , the other must make the same decision by Lemma 11 and Lemma 14. Otherwise, both decisions are indirect and derived from a later committed leader. Let k_i and k_j be the rounds of the first such commits for v_i and v_j , respectively. By the induction hypothesis, $k_i = k_j$, and both validators commit the same leader block. Since indirect decisions depend only on the causal history of that block, both validators derive the same decision for B_L^k . $\square$

THEOREM 7 (CONSENSUS SAFETY). *In Mysticeti-Beluga, all honest validators order transactions consistently.*

PROOF. By Lemma 16, all honest validators decide a consistent status for each round leader block, meaning that all honest validators decide identical *to-commit* leader blocks. According to the consensus logic employed by Mysticeti-Beluga, all honest validators will order *to-commit* leader blocks and their causal history block consistently. Therefore, for transactions included in the ordered blocks, all honest validators order them consistently. $\square$

E The Pseudocode of the AC-based Push Protocol

Figure 8 provides a pseudocode for Beluga's push protocol. The key components consist of a reputation mechanism (lines 23-32) and an admission control (lines 1-22), both of which are detailed in Section 4.2.

F Beluga Implementation

We implement Beluga in Rust within Mysticeti [6] by forking the Mysticeti codebase [27]. It leverages tokio [46] for asynchronous networking, utilizing raw TCP sockets for communication, implementing reliable point-to-point channels, necessary to correctly implement the distributed system abstractions without relying on any RPC frameworks. For cryptographic operations, it relies on ed25519-consensus [18] for asymmetric cryptography and blake2 [33] for cryptographic hashing. To ensure data persistence and crash recovery, it employs a Write-Ahead Log (WAL) optimizing I/O operations through vectored writes [20], efficient memory-mapped files, and uses minibytes [28] to minimize copies and serialization.

Variables:

R_L – The score decrease each time
 $TR_i[]$ – An array of reputations (indexed by validators)
 struct block B
 ... ▷ original fields
 $B.weaklinks$ - used to link blocks that v_i has received and accepted but not referenced as parents
 $B.watermark[]$ - an array of the highest round numbers of all validators' blocks received by v_i
 $B.ancestors[]$ - an array of the highest round numbers of all validators' blocks reachable from B
 ▷ Call $block_propose_i(B, r)$ to push a round r block B
 1: **procedure** $create_new_block(r, \mathcal{B}^{r-1})$ ▷ $\mathcal{B}^{r-1}$ is a list of the latest received blocks from all validators with round $\leq r-1$
 2: Initializes a block B with $B.r = r$, $B.author = i$, and other original fields
 3: $parents \leftarrow AC_parent_selection(r, \mathcal{B}^{r-1})$
 4: $B.parents \leftarrow$ digests of $parents$
 5: $B.weaklinks \leftarrow \{B'.d \mid B' \in \mathcal{B}^{r-1} \setminus parents \text{ is acceptable}\}$
 6: $watermark \leftarrow []$
 7: **for** $\forall B' \in \mathcal{B}^{r-1}$ **do**
 8: $watermark[B'.author] \leftarrow B'.r$
 9: $B.watermark \leftarrow watermark$
 10: $B.ancestors \leftarrow compute_ancestors(parents)$
 11: signs and broadcasts B using best-effort broadcast
 12: update $score_with_watermarks(r, \mathcal{B}^{r-1})$
 13: outputs $block_accept_i$ and $block_store_i$ for B and every acceptable block in $\mathcal{B}^{r-1}$, if it hasn't done so already
 14: **procedure** $AC_parent_selection(r, \mathcal{B})$
 15: $\mathcal{B} \leftarrow \{B' \in \mathcal{B} \mid B'.r = r-1 \wedge B' \text{ is acceptable}\}$
 16: $parents \leftarrow$ top $2f+1$ blocks in $\mathcal{B}$ by $TR_i[B'.author]$
 17: **return** $parents$
 18: **procedure** $compute_ancestors(parents)$
 19: $ancestors \leftarrow []$
 20: **for** $k \in 1, \dots, n$ **do**
 21: $ancestors[k] \leftarrow \max(\{B'.r \mid B' \in parents \wedge B'.author = v_k\} \cup \{B'.ancestors[k] \mid B' \in parents\})$
 22: **return** $ancestors$
 23: **procedure** $update_score_with_watermarks(r, \mathcal{B})$
 24: **for** $j \in 1, \dots, n$ **do**
 25: $count \leftarrow 0$
 26: **for** $\forall B' \in \mathcal{B}$ **do**
 27: **if** $B'.watermark[j] == r-2$ **then**
 28: $count \leftarrow count + 1$
 29: **if** $count \geq 2f+1$ **then**
 30: $TR_i[j] \leftarrow TR_i[j] + 1$
 31: **upon** pulling or receiving $f+1$ pull requests (i.e., blames) for a missing block created by v_j **do**
 32: $TR_i[j] \leftarrow TR_i[j] - R_L$

Figure 8: Beluga's AC-based optimistic push protocol for validator v_i .

By default, this Mysticeti implementation uses a traditional optimistic push followed by a random pull protocol (described in Section 2). We modify its block synchronizer module to use Beluga instead. Implementing our mechanism requires adding less than 200 LOC, and does not require any extra cryptographic tools.

In addition to regular unit tests, we inherited and utilized two supplementary testing utilities from the Mysticeti codebase. First, a simulation layer replicates the functionality of the *tokio* runtime and TCP networking. This simulated network accurately simulates real-world WAN latencies, while the *tokio* runtime simulator employs a discrete event simulation approach to mimic the passage of time. Second, a command-line utility (called *orchestrator*) which deploys real-world clusters of Beluga on machines distributed across the globe.

We are open-sourcing our Beluga implementation, along with its simulator and orchestration tools, to ensure reproducibility of our results⁷.

G Experimental Setup

This section describes the experimental setup used for evaluating Beluga in Section 6.

We deploy all systems on AWS, using *m5d.8xlarge* instances across 13 different AWS regions: Northern Virginia (us-east-1), Oregon (us-west-2), Canada (ca-central-1), Frankfurt (eu-central-1), Ireland (eu-west-1), London (eu-west-2), Paris (eu-west-3), Stockholm (eu-north-1), Mumbai (ap-south-1), Singapore (ap-southeast-1), Sydney (ap-southeast-2), Tokyo (ap-northeast-1), and Seoul (ap-northeast-2). Validators are distributed across those regions as equally as possible. Each machine provides 10 Gbps of bandwidth, 32 virtual CPUs (16 physical cores) on a 3.1 GHz Intel Xeon Skylake 8175M, 128 GB memory, and runs Linux Ubuntu server 24.04. We select these machines because they provide decent performance, are in the price range of “commodity servers”, and match the minimal specifications of modern quorum-based blockchains [45].

The *latency* refers to the time elapsed from the moment a client submits a transaction to when it is committed by the validators, and the *throughput* refers to the number of transactions committed per second. We instantiate several geo-distributed benchmark clients within each validator submitting transactions in an open loop model, at a fixed rate. We experimentally increase the load of transactions sent to the systems, and record the throughput and latency of commits. As a result, all plots illustrate the steady-state latency of all systems under various loads. Transactions in the benchmarks are arbitrary and contain 512 bytes. We configure both systems with 2 leaders per round, and a leader timeout of 1 second.

⁷ <https://anonymous.4open.science/r/beluga-0F4F>